

Contents

TotalWaitingTime	3
DominoSequence	4
LongestTileSequence	5
DivideIntoGroups	6
BalanceTheBoard	7
SimilarRectangles	8
SameDigitEnds	9
SkyscrapersHeights	10
CarAssembly	11
TwoProjects	12
MonthlyBalance	13
GraphDisappearance	14
LongestPath	17
LatestAmbulanceArrival	19
MinStackHeightSum	22
GreenNeighborhood	24
EndsTheSame	25
AssassinsStealth	26
FreeingStorageSpace	27
NoEqualDigits	28
SkillTree	29
RoadRoller	31
RoadRepair	33
GardenArrangement	34
TwoWayRoadRenovation	35
RecyclingTrucks	36
VaryingPathInTree	38
DivisibleByThree	40
PathDetection	40
EqualSegments	42
MinDistinct	43
LongestEvenCount	44
LargestPalindrome	44

DeleteDefectiveRows.....	45
AngryFrogs.....	47
TwoRooks.....	49
CoverMaximalPoints.....	51
Sticks.....	52
FairSplits.....	52
SquareTiles.....	53
WordSplit.....	54
ReverseEdges.....	55
RedSegment.....	58
EraseAdjacentLetters.....	59
FourDigitsPath.....	60
CandyShops.....	61
ArraySlicing.....	62
Basketballs.....	63
OptimizingFarthestDifferent.....	64
MaxBiValuedSlice.....	65
Times17DecDigitsSum.....	65
IntegerReductionCount.....	66
OnesInArithSeq.....	67

TotalWaitingTime

There are N clients who have ordered N handmade items. The K -th client ordered exactly one item that takes $T[K]$ hours to make. There is only one employee who makes items for clients, and they work in the following manner:

- spend one hour making the first item;
- if the item is finished, the employee delivers it to the client immediately;
- if the item is not finished, they put it after the N -th item for further work;
- the employee starts making the next item.

For example for $T = [3, 1, 2]$, the employee spends 6 hours making items in the following order: $[1, 2, 3, 1, 3, 1]$. The first client waited 6 hours for their item, the second client received their item after 2 hours and the third client after 5 hours. What is the total time that clients need to wait for all ordered items? For the above example, the answer is $6 + 2 + 5 = 13$.

As the result may be large, return its last nine digits without leading zeros (in other words, return the result modulo 10^9).

Write a function:

```
def solution(T)
```

that, given an array of integers T of length N , returns the total time that the clients need to wait (modulo 10^9).

Examples:

1. Given $T = [3, 1, 2]$, the function should return 13 as explained above.
2. Given $T = [1, 2, 3, 4]$, the function should return 24. The employee prepares the items in the following order: 1, 2, 3, 4, 2, 3, 4, 3, 4, 4. The first client waited for 1 hour, the second client for 5 hours, the third client for 8 hours, and the fourth client for 10 hours. The total waiting time of all clients is $1 + 5 + 8 + 10 = 24$ hours.
3. Given $T = [7, 7, 7]$, the function should return 60.
4. Given $T = [10000]$, the function should return 10000.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array T is an integer within the range $[1..10,000]$.

DominoSequence

A domino is a rectangular tile divided into two square parts. There are between 1 and 6 dots on each of the parts.

There is an array A of length $2*N$, representing N dominoes. Dominoes are arranged in a line in the order in which they appear in array A. The number of dots on the left and the right parts of the K-th domino are $A[2*K]$ and $A[2*K+1]$, respectively. For example, an array $A = [2, 4, 1, 3, 4, 6, 2, 4, 1, 6]$ represents a sequence of five domino tiles: (2, 4), (1, 3), (4, 6), (2, 4), and (1, 6).



In a *correct domino sequence*, each pair of neighboring tiles should have the same number of dots on their adjacent parts. For example, tiles (2, 4) and (4, 6) form a correct domino sequence and tiles (2, 4) and (1, 3) do not.

What is the minimum number of domino tiles that must be removed from the sequence so that the remaining tiles form a correct domino sequence? It is not allowed to reorder or rotate the dominoes.

Write a function:

```
def solution(A)
```

that, given an array A representing a sequence of N domino tiles, returns the minimum number of tiles that must be removed so that the remaining tiles form a *correct domino sequence*.

Examples:

1. Given $A = [2, 4, 1, 3, 4, 6, 2, 4, 1, 6]$, the function should return 3. The second and the last two dominoes should be removed. After this, the remaining dominoes are (2, 4) and (4, 6).



2. Given $A = [5, 1, 2, 6, 6, 1, 3, 1, 4, 3, 4, 3, 4, 6, 1, 2, 4, 1, 6, 2]$, the function should return 7. The domino tiles that should remain are: (2, 6), (6, 1), (1, 2).

3. Given $A = [1, 5, 3, 3, 1, 3]$, the function should return 2. No pair of dominoes can be connected without rotating or reordering them.

4. Given $A = [3, 4]$, the function should return 0.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..50,000];
- the length of array A is equal to 2*N;
- each element of array A is an integer within the range [1..6].

LongestTileSequence

There are N tiles (numbered from 0 to $N-1$). Each tile is made of two squares that are colored either red (represented by the letter "R") or green (represented by "G"). A tile is described by a two-character string representing the respective colors of the left and right squares. The tiles cannot be rotated (which means that "RG" and "GR" tiles are different). Two tiles can be placed next to each other if the color of their adjacent squares is the same.

What is the length of the longest possible sequence that can be created using the provided tiles?

Write a function:

```
def solution(A)
```

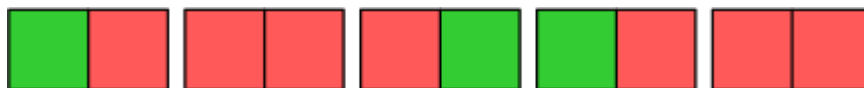
that, given an array A of N strings, representing the tiles, returns the maximum number of tiles that can be arranged in a sequence.

Examples:

1. Given $A = ["RR", "GR", "RG", "GR", "GR", "RR"]$,

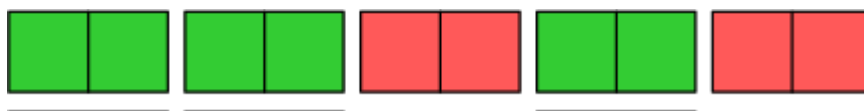


we can select tiles 0, 2, 3, 4, 5 (underlined in the picture above) and arrange them into the sequence GR - RR - RG - GR - RR:



The function should return 5.

2. Given $A = ["GG", "GG", "RR", "GG", "RR"]$,



we can select tiles 0, 1, 3 (underlined in the picture above) and arrange them into the sequence GG - GG - GG:



The function should return 3.

3. Given A = ["RG", "GR", "RG", "GR"],



all tiles can be used without reordering them. The function should return 4.

4. Given A = ["RG", "RG", "RG"], the function should return 1.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each string in array A is one of the following: "RR", "RG", "GR", "GG".

DivideIntoGroups

There is an array A consisting of N integers. Divide them into three non-empty groups. In each group we calculate the difference between the largest and smallest integer. Our goal is to make the maximum of these differences as small as possible.

For example, given A = [11, 5, 3, 12, 6, 8, 1, 7, 4], we can divide the elements into three groups:

- [3, 1, 4] - the difference between elements is 3;
- [5, 6, 8, 7] - the difference is also 3;
- [11, 12] - the difference is 1.

The maximum difference equals 3, which is the minimum possible result.

Write a function:

```
def solution(A)
```

that, given an array A, returns the minimum possible result as explained above.

Examples:

1. For A = [11, 5, 3, 12, 6, 8, 1, 7, 4], the function should return 3, as explained above.

2. For A = [10, 14, 12, 1000, 11, 15, 13, 1], the function should return 5. The elements of A should be divided into three groups as follows:

- [1];
- [10, 14, 12, 11, 15, 13];
- [1000].

3. For $A = [4, 5, 7, 10, 10, 12, 12, 12]$, the function should return 2. The elements of A could be divided into these three groups:

- [4, 5];
- [7];
- [10, 10, 12, 12].

4. For $A = [5, 10, 10, 5, 5]$, the function should return 0. The first group may contain all elements with value 5; the second and the third groups may each contain one element with value 10.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[3..100,000]$;
- each element of array A is an integer within the range $[0..1,000,000,000]$.

BalanceTheBoard

There is a board made of two rows and N columns. The board is represented by two strings, `row1` and `row2`, made of characters 'R', 'W' and/or '?'. A board is *balanced* if, in each row and in each column, the number of characters 'R' is equal to the number of characters 'W'. For example, the following board is balanced:

```
?RW?WR
?WR?RW
```

and the following board is not balanced:

```
W?WR?
R??W?
```

(there are two characters 'W' and one character 'R' in the first row).

The question marks ('?') can be replaced with 'W' or 'R'. What is the minimum number of replacements needed to balance the board?

Write a function:

```
def solution(row1, row2)
```

that, given two strings `row1` and `row2` made of N characters each, returns the minimum number of replacements needed to balance the board. If it is not possible to balance the board, the function should return -1.

Examples:

1. For row1 = "W?WR?" and row2 = "R??W?", the board initially looks as follows:

```
W?WR?  
R??W?
```

The last question mark in the first row and the second question mark in the second row can be replaced with 'R'. The last question mark in the second row can be replaced with 'W'. After these replacements, the board is balanced and looks as follows:

```
W?WRR  
R?RWW
```

The function should return 3.

2. For row1 = "R?R??" and row2 = "??W??", five replacements are needed to balance the board:

```
R?RWW  
W?WRR
```

The function should return 5.

3. For row1 = "RR?" and row2 = "?R?", it is not possible to balance the board. The function should return -1.

4. For row1 = "WWW??" and row2 = "RRR??", it is not possible to balance the board. The function should return -1.

5. For row1 = "?RW?WR" and row2 = "?WR?RW", the board is already balanced. The function should return 0.

Write an efficient algorithm for the following assumptions:

- row1 and row2 are made only of the characters 'W', 'R' and/or '?';
- row1 and row2 have the same length, equal to N;
- N is an integer within the range [1..100,000].

SimilarRectangles

There are n rectangles numbered from 0 to $n-1$. The K -th rectangle has size $A[K] \times B[K]$.

We call two rectangles *similar* when:

- they share a side of the same length, and
- the length of the other sides differs by at most 1.

For example:

- rectangles 2×3 and 2×4 are similar,
- rectangles 2×3 and 4×2 are also similar (rectangles can be rotated),
- rectangles 2×3 and 2×5 are not similar (other sides differ by 2),
- rectangles 2×2 and 3×3 are not similar either (more than one side differing in length).

The task is to choose as many rectangles as possible in such a way that every two chosen rectangles are similar to each other. Note that rectangles can be rotated.

Write a function:

```
int solution(int A[], int B[], int N);
```

that, given two arrays A and B of N integers each, returns the maximum number of similar rectangles that can be chosen.

Examples:

1. Given A = [500, 500] and B = [500, 500], the function should return 2. Both rectangles can be chosen.
2. Given A = [5, 5, 1, 6, 4] and B = [2, 1, 6, 1, 1] the function should return 3. The best choice is to take the rectangles in positions 1 to 3. After rotation, their sizes are respectively 5×1, 6×1 and 6×1.
3. Given A = [3, 3, 3, 3, 5, 7, 1, 1] and B = [4, 6, 8, 10, 10, 10, 2, 3] the function should return 2. The optimal choice is to take last two rectangles (sizes 1×2 and 1×3).
4. Given A = [4, 4, 2, 2, 2, 4, 4] and B = [3, 4, 6, 6, 6, 3, 4] the function should return 4. The best choice is to take the first two and the last two rectangles (sizes 3×4, 4×4, 3×4 and 4×4).

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..200,000];
- arrays A and B consist of N integers within the range [1..500].
-

SameDigitEnds

There is an array A consisting of N integers. What is the maximum sum of two integers from A that share their first and last digits? For example, 1007 and 167 share their first (1) and last (7) digits, whereas 2002 and 55 do not.

Write a function:

```
def solution(A)
```

that, given an array A consisting of N integers, returns the maximum sum of two integers that share their first and last digits. If there are no two integers that share their first and last digits, the function should return -1 .

Examples:

1. Given $A = [130, 191, 200, 10]$, the function should return 140. The only integers in A that share first and last digits are 130 and 10.
2. Given $A = [405, 45, 300, 300]$, the function should return 600. There are two pairs of integers that share first and last digits: (405, 45) and (300, 300). The sum of the two 300s is bigger than the sum of 405 and 45.
3. Given $A = [50, 222, 49, 52, 25]$, the function should return -1 . There are no two integers that share their first and last digits.
4. Given $A = [30, 909, 3190, 99, 3990, 9009]$, the function should return 9918.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array A is an integer within the range $[10..1,000,000,000]$.

SkyscrapersHeights

You are given an array A of N integers, representing the maximum heights of N skyscrapers to be built.

Your task is to specify the actual heights of the skyscrapers, given that:

- the height of the K -th skyscraper should be positive and not bigger than $A[K]$;
- no two skyscrapers should be of the same height;
- the total sum of the skyscrapers' heights should be the maximum possible.

Write a function:

```
def solution(A)
```

that, given an array A of N integers, returns an array B of N integers where $B[K]$ is the assigned height of the K -th skyscraper satisfying the above conditions.

If there are several possible answers, the function may return any of them. You may assume that it is always possible to build all skyscrapers while fulfilling all the requirements.

Examples:

1. Given $A = [1, 2, 3]$, your function should return $[1, 2, 3]$, as all of the skyscrapers may be built to their maximum height.

2. Given $A = [9, 4, 3, 7, 7]$, your function may return $[9, 4, 3, 7, 6]$. Note that $[9, 4, 3, 6, 7]$ is also a valid answer. It is not possible for the last two skyscrapers to have the same height. The height of one of them should be 7 and the other should be 6.

3. Given $A = [2, 5, 4, 5, 5]$, your function should return $[1, 2, 3, 4, 5]$.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..50,000]$;
- each element of array A is an integer within the range $[1..1,000,000,000]$;
- there is always a solution for the given input.

CarAssembly

In a car factory there are two assembly lines, called A and B . Each of them consists of N positions numbered from 0 to $N-1$. In each position, some component is installed into the car being assembled. The lines are symmetric; that is, the same component is installed into the car in the corresponding positions of A and B .

In order to assemble a car, it needs to get through all of the positions on the assembly lines. It takes $A[K]$ minutes to install the component in the K -th position on line A , and $B[K]$ minutes to install it on line B .

After installing a component in the K -th position, the car can either be moved to the next position on the same line (which happens automatically and takes 0 minutes) or it can be moved to the other line. Switching from line A to line B takes X minutes, and switching from line B to line A takes Y minutes.

The assembly process can be started in either line.

Write a function:

```
def solution(A, B, X, Y)
```

that, given arrays A and B consisting of N integers each, and integers X and Y, returns the minimum time required to assemble an entire car.

Examples:

1. Given A = [1, 6, 2], B = [3, 2, 5], X = 2 and Y = 1, the function should return 8. The assembly of the car can start in line A. After installing the component in position number 0 (1 minute), the car can be moved to assembly line B (2 minutes) to install the component in position number 1 (2 minutes), and move the car back to line A (1 minute), where the last component would be installed (2 minutes). The total time is $1 + 2 + 2 + 1 + 2 = 8$.
2. Given A = [2, 11, 4, 4], B = [9, 2, 5, 11], X = 8 and Y = 4, the function should return 21. It is optimal to assemble the entire car on line A.
3. Given A = [1, 10, 1], B = [10, 1, 10], X = 1 and Y = 5, the function should return 9.
4. Given A = [8, 3, 3], B = [6, 1, 10], X = 4 and Y = 3, the function should return 13.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each element of arrays A and B is an integer within the range [1..20,000];
- X and Y are integers within the range [1..20,000].

TwoProjects

A company is planning N projects, numbered from 0 to N-1. Completing the K-th project will bring value V[K] to the company. For some projects there may be additional requirements - the L-th requirement states that before starting project B[L], project A[L] should be completed. There are M such requirements.

The company has enough resources for at most two projects to be completed. If two projects are chosen, they will be completed one by one (in sequential order) and the total value they bring to the company is the sum of their individual values. What is the highest value that a valid choice of projects can bring to the company?

Write a function:

```
int solution(int V[], int N, int A[], int B[], int M);
```

that, given array V of N integers and two arrays A and B of M integers each, returns the maximum value that the company may gain by completing at most two possible projects.

Examples:

1. Given $V = [-3, 5, 7, 2, 3]$, $A = [3, 1]$ and $B = [2, 4]$, the function should return 9. This can be achieved by completing project 3 (with value 2) first and then project 2 (with value 7).

2. Given $V = [1, 1, 5]$, $A = [0, 1]$ and $B = [2, 2]$, the function should return 2. This can be achieved by completing projects 0 and 1. Project 2 can not be completed as it requires both projects 0 and 1 completed before.

3. Given $V = [5, 6, 6, 7, -10]$, $A = [0, 0, 0, 1, 2, 3]$ and $B = [1, 2, 3, 3, 1, 2]$, the function should return 5. The projects that are possible to be completed are 0 and 4. As project 4 would bring negative value to the company, it is optimal to do only project 0. The structure of dependencies of projects 1, 2 and 3 form a cycle, so none of them can be completed in a valid choice of projects.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- M is an integer within the range $[0..100,000]$;
- each element of array V is an integer within the range $[-1,000,000,000..1,000,000,000]$;
- each element of arrays A and B is an integer within the range $[0..N-1]$;
- a project may not require itself to be completed ($A[K] \neq B[K]$);
- projects' requirements do not repeat.

MonthlyBalance

You are given two arrays A and B of N strings each, and an array C of N positive integers. They describe N monthly expenses (numbered from 0 to $N-1$) set up on an account. The K -th expense charges the account with an amount $C[K]$ at the end of each month, starting from month $A[K]$ and ending in month $B[K]$ (inclusive).

The format of each string is "MM-YYYY" (where MM denotes month, between 01 and 12, and YYYY denotes year, between 1900 and 2100), for example: "06-2021".

What is the minimum monthly income needed for all of these charges to be paid?

We cannot go into debt. In other words, the amount of money that remains in the account at the end of each month (after all the payments) should not drop below 0. The monthly income starts in the earliest month given in array A. The income is transferred onto the account at the beginning of the month, i.e. income for a given month is always on the account before any of the charges are paid. The balance that remains in the account at the end of the month can be used in the following months.

Write a function:

```
def solution(A, B, C)
```

that, given arrays A and B consisting of N strings each and an array C consisting of N positive integers, representing the expenses as described above, returns the minimum monthly income needed to cover all of the expenses.

Examples:

1. Given A = ["03-2021", "04-2021", "05-2021"], B = ["03-2021", "05-2021", "05-2021"], and C = [20, 10, 15], the function should return 20. The total expenses are: 20 (in March), 10 (in April) and 25 (in May). Starting with income 20 (in March), the total balance at the end of each month will be: 0 (in March), 10 (in April) and 5 (in May).

2. Given A = ["10-2020", "01-2020", "02-2020", "06-2021"], B = ["07-2021", "03-2020", "10-2020", "07-2021"], and C = [1, 10, 2, 90], the function should return 13.

3. Given A = ["01-1900", "12-2099", "11-2099", "01-1901"], B = ["12-1901", "12-2099", "12-2100", "01-1902"] and C = [1, 1000, 998, 1], the function should return 7.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each element of array C is an integer within the range [1..1,000];
- all dates in arrays A and B are correct dates in the format "MM-YYYY" between "01-1900" and "12-2100";
- A[K] is a date not later than B[K] for each K from 0 to N-1

GraphDisappearance

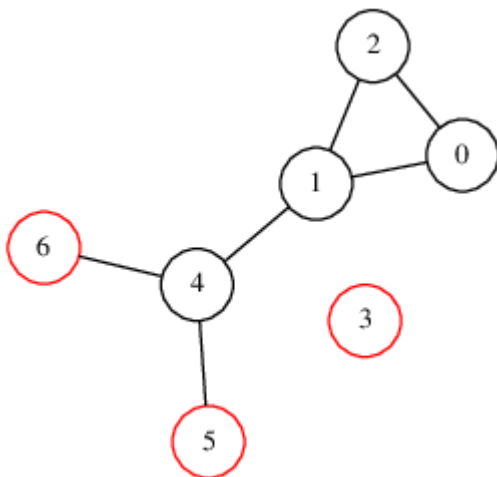
You are given an undirected graph consisting of N vertices, numbered from 0 to N-1, connected with M edges. The graph is described by two arrays, A and

B, both of length m . A pair $(A[K], B[K])$, for K from 0 to $m-1$, describes an edge between vertex $A[K]$ and vertex $B[K]$.

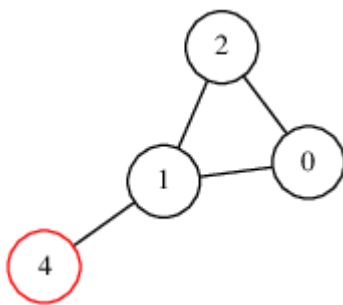
Each second, every vertex with at most one edge connected to it disappears. Every edge which is connected to one of the disappearing vertices also disappears.

After how many seconds will the vertices stop disappearing?

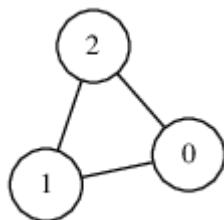
Consider a graph with $N = 7$ vertices and following 6 edges: $(0, 1)$, $(1, 2)$, $(2, 0)$, $(1, 4)$, $(4, 5)$ and $(4, 6)$.



After the first second, vertices 3, 5, and 6 (marked red in the picture above) will disappear:



After the next second vertex 4 will disappear and only vertices 0, 1 and 2 will be left:



All three remaining vertices are connected to two edges, so none of will ever disappear and the answer is 2.

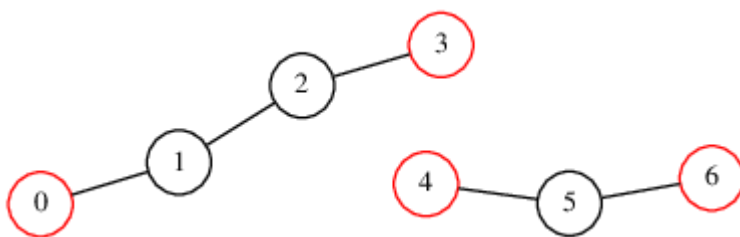
Write a function:

```
int solution(int N, int A[], int B[], int M);
```

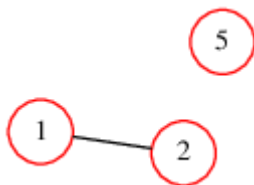
that, given an integer N and two arrays A and B of M integers each, returns the number of seconds after which the vertices will stop disappearing, or 0 if no vertices will ever disappear.

Examples:

1. Given $N = 7$, $A = [0, 1, 2, 1, 4, 4]$ and $B = [1, 2, 0, 4, 5, 6]$, the function should return 2, as explained above.
2. Given $N = 7$, $A = [0, 1, 2, 4, 5]$ and $B = [1, 2, 3, 5, 6]$, the function should return 2. The graph from this example is shown below:

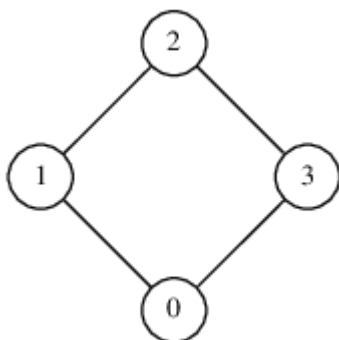


After the first second, vertices 0, 3, 4, and 6 (marked red in the picture above) will disappear:

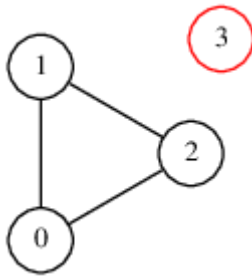


During the next second, all of the remaining vertices disappear, so the answer is 2.

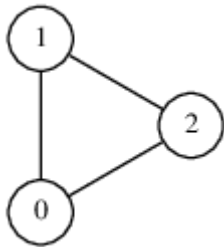
3. Given $N = 4$, $A = [0, 1, 2, 3]$ and $B = [1, 2, 3, 0]$, the function should return 0. Each vertex is connected with two edges, so none of them will disappear.



4. Given $N = 4$, $A = [0, 1, 2]$ and $B = [1, 2, 0]$, the function should return 1.



After the first second, vertex 3 (marked red in the picture above) will disappear. Other vertices are connected with two edges and will never disappear, so the answer is 1.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[2..100,000]$;
- M is an integer within the range $[1..100,000]$;
- all elements of arrays A and B are integers within the range $[0..N-1]$;
- there are no self-loops (edges with $A[K] = B[K]$) in the graph;
- there are no multiple edges between the same vertices.

LongestPath

A group of explorers is heading for an expedition into a cave. We know that the cave can be represented as a rectangular map consisting of N rows and M columns. Rows represent levels of the cave and are numbered from 0 to $N-1$ from top to bottom. Columns are numbered from 0 to $M-1$ from left to right and represent consecutive chambers on each level of the cave. Explorers can get through some chambers (marked '.') without any problem, but some other chambers (marked 'X') are impossible to pass through as they contain too much debris and are unsafe.

The expedition starts at the top-left corner $(0, 0)$ of the cave and its goal is to get to the exit in the bottom-right corner $(N-1, M-1)$. Explorers can move only to the chamber on the left, the chamber on the right or climb down to the lower level. Climbing up is too dangerous, so there is no possibility

to move to the upper level after moving to a lower one. Formally, if the expedition is in chamber (R, C) , the next chamber on their path may be only one of the following: $(R+1, C)$, $(R, C-1)$, $(R, C+1)$.

The explorers want to plan their path through the cave to explore as many chambers as possible. As the chambers may be very narrow, they do not want to be forced to move back during the expedition. In other words, the explorers do not want to enter a chamber they have already been in before.

Chambers outside the map are considered unsafe. The expedition can never enter any unsafe chamber.

What is the maximum number of chambers the explorers can visit during the expedition? If it would be impossible to reach chamber $(N-1, M-1)$, for reasons of safety, the expedition should be canceled.

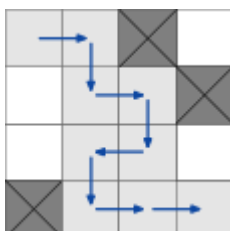
Write a function:

```
def solution(B)
```

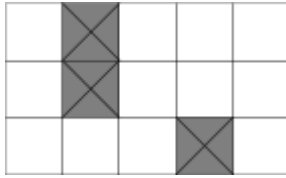
that, given an array B consisting of N strings of equal length M , representing the map of the cave, returns the maximum number of chambers the expedition can explore on its way from chamber $(0, 0)$ to the exit in chamber $(N-1, M-1)$. If there is no possibility for the expedition to get to the chamber with the exit, the function should return -1 .

Examples:

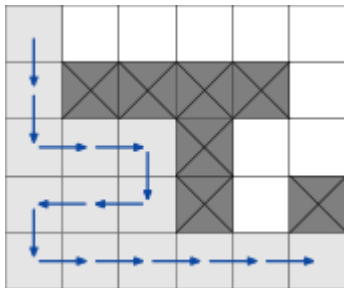
1. Given $B = [".X.", "...X", "...", "X..."]$, your function should return 9. One of the longest paths through the cave is shown in the following image.



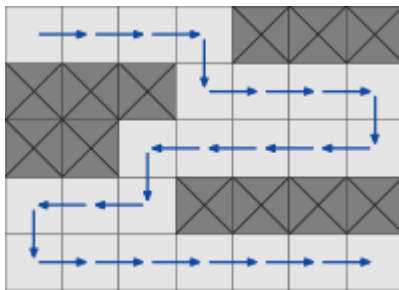
2. Given $B = [".X...", ".X...", "...X."]$, your function should return -1 . The explorers would not be able to reach the exit chamber as it would require climbing to an upper level of the cave.



3. Given $B = [\text{"....."}, \text{"XXXX."}, \text{"...X.."}, \text{"...X.X"}, \text{"....."}]$, your function should return 14. The longest path is unique:



4. Given $B = [\text{"....XXX"}, \text{"XXX...."}, \text{"XX....."}, \text{"...XXXX"}, \text{"....."}]$, your function should return 23. It is possible for the explorers to wander through all safe chambers of the cave.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..400]$;
- each string from array B is of length M , which is an integer within the range $[1..400]$;
- each string from array B consists of characters: 'X' and/or '.';
- top-left and bottom-right corner chambers are safe.

LatestAmbulanceArrival

In the city there are N districts (numbered from 0 to $N-1$) connected with M streets. The connections are described by two arrays, A and B , both of length M . A pair $(A[K], B[K])$ marks a street between districts $A[K]$ and $B[K]$ (for K from 0 to $M-1$). There are also L hospitals whose locations are described by an array H . The J -th hospital is placed in district $H[J]$ (for J from 0 to $L-1$).

If an ambulance is needed in a district, one is sent from the hospital from which it will arrive in the shortest time. The ambulance arrives by the shortest possible route; passing one street takes it exactly 1 minute.

Potentially, there might be a patient in need in any district of the city. What is the longest time required to reach any possible patient with an ambulance?

Write a function

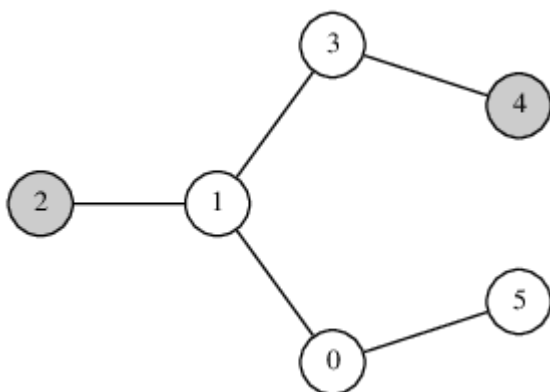
```
def solution(N, A, B, H)
```

that, given an integer N , arrays A and B describing streets in the city and an array H giving locations of hospitals, returns an integer - the longest driving time (in minutes) between a district and its closest hospital. If some district cannot be reached with an ambulance, the function should return -1 .

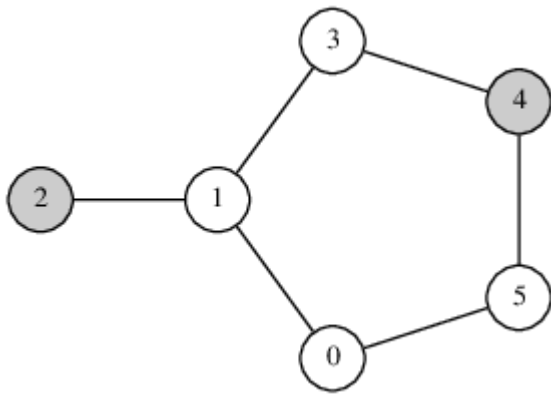
Examples:

1. Given $N = 6$, $A = [0, 1, 1, 3, 0]$, $B = [1, 2, 3, 4, 5]$, and $H = [2, 4]$, your function should return 3. District number 5 has the longest waiting time. The ambulance will arrive from the hospital in district 2 in three minutes via route $2 \rightarrow 1 \rightarrow 0 \rightarrow 5$.

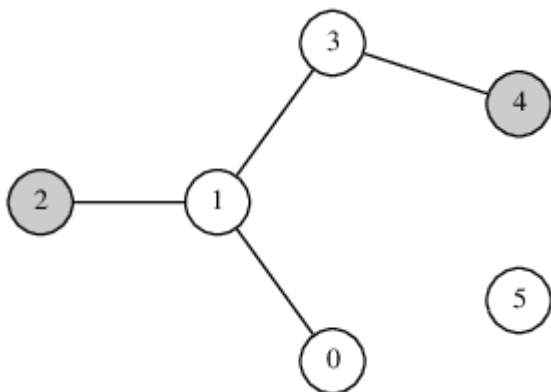
The city is shown in the diagram below. Districts with hospitals are shaded in gray.



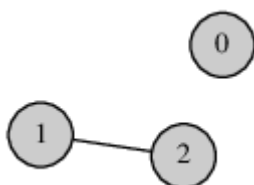
2. Given $N = 6$, $A = [0, 1, 1, 3, 0, 4]$, $B = [1, 2, 3, 4, 5, 5]$, and $H = [2, 4]$, your function should return 2. Now the district with the longest ambulance arrival time is district 0. Every other district can be reached by an ambulance in at most 1 minute.



3. Given $N = 6$, $A = [0, 1, 1, 3]$, $B = [1, 2, 3, 4]$, and $H = [2, 4]$, your function should return -1. District 5 is not connected with any district with a hospital.



4. Given $N = 3$, $A = [1]$, $B = [2]$, and $H = [0, 1, 2]$, your function should return 0. There is a hospital in every district.



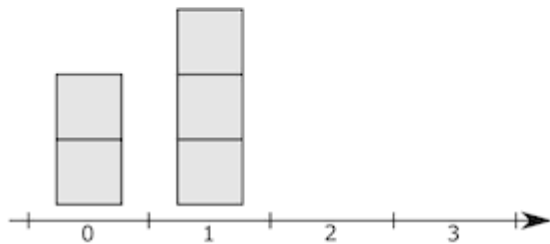
Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- M is an integer within the range $[0..100,000]$;
- L is an integer within the range $[1..N]$;
- the elements of H are all distinct;
- each element of arrays A , B and H is an integer within the range $[0..N-1]$;
- every street goes between two different districts;
- there are no multiple streets between two districts.

MinStackHeightSum

Let us consider an infinite sequence of stacks indexed from 0, and an exchange operation that removes two tokens from a stack and adds one token to the next stack.

For example, let's assume there are two tokens on stack 0 and three on stack 1. Two tokens from stack 0 may be exchanged for one new token on stack 1. After that operation, there are four tokens on stack 1 that may be exchanged for two new tokens on stack 2. Finally, a new token may be added to stack 3 by exchanging two tokens from stack 2. This gives us: stacks 0, 1 and 2 empty, and stack 3 with one token.



Given the heights of the first N stacks, find the minimum number of tokens that may remain after any number of exchange operations. You may assume that all of the tokens are identical. All uninitialized stacks are empty by default.

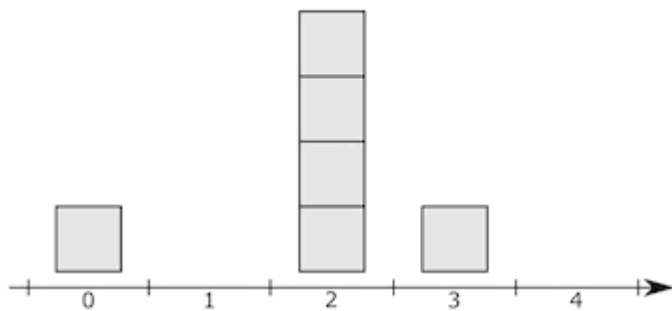
Write a function:

```
def solution(A)
```

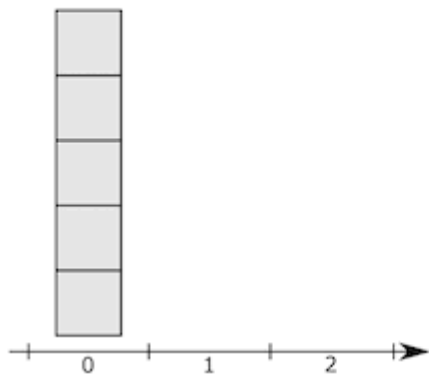
that, given an array A of N integers, representing the heights of the first N stacks in the sequence, returns the minimum number of tokens which may remain on the stacks after any number of exchange operations.

Examples:

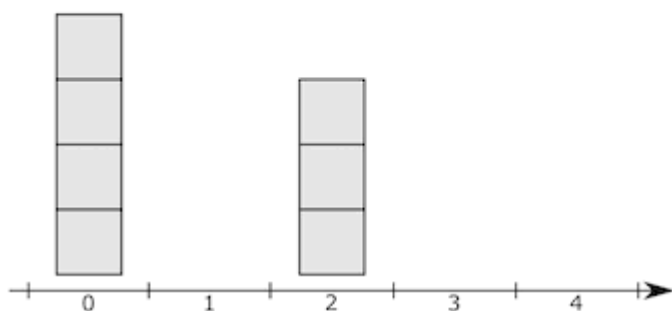
1. Given $A = [2, 3]$, the function should return 1, as explained above.
2. Given $A = [1, 0, 4, 1]$, the function should return 3. One token from stack 0 cannot be exchanged. Then the four tokens from stack 2 can be exchanged for two tokens on stack 3. In the end, two of the three tokens from stack 3 can be exchanged for one token on stack 4. That gives us three tokens, one on each of the stacks 0, 3 and 4.



3. Given $A = [5]$, the function should return 2. Four of the five tokens from stack 0 can be exchanged for two on stack 1, and then in turn for one token on stack 2. After that we have one token on each of stacks 0 and 2.



4. Given $A = [4, 0, 3, 0]$, the function should return 1. The four tokens from stack 0 can be exchanged for two on stack 1, then for one token on stack 2. The four tokens that are now on stack 2 can be exchanged for two on stack 3, then for one token on stack 4.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array A is an integer within the range $[0..1,000,000]$.

GreenNeighborhood

We are provided with a plan of an industrial village, represented by an array A consisting of N integers. The K -th value (for K within the range $0 \dots N-1$) represents a field which may contain:

- a forest consisting of $A[K]$ trees (if $A[K]$ is positive); or
- an industrial building (if $A[K]$ is non-positive), producing $-A[K]$ units of pollution.

One tree is able to neutralize one unit of pollution. Our goal is to make every neighborhood sustainable, i.e. for every field, the sum of its value and the values of its neighbors (adjacent fields to the left and right) should be greater than or equal to zero. To achieve this goal, we can plant additional trees in any chosen field (note that we can plant trees in fields containing industrial buildings).

For example, given $A = [1, -3, 2]$, there is one tree in the field number 0, an industrial building producing 3 units of pollution in field number 1 and two trees in field number 2. The sums of values of the fields and their neighbors are $1 + (-3) = -2$ for field number 0, $1 + (-3) + 2 = 0$ for field number 1, and $(-3) + 2 = -1$ for field number 2. The neighborhoods of fields 0 and 2 are not sustainable, as their sums are negative. After planting two trees in field 1, we obtain $A = [1, -1, 2]$. In the new array, the sums are respectively 0, 2, and 1, which makes every neighborhood sustainable.

What is the minimum number of trees we have to plant in order to make every field's neighborhood sustainable?

Write a function:

```
def solution(A)
```

that, given an array A consisting of N integers, returns the minimum number of trees we have to plant in order to make every field's neighborhood sustainable.

Examples:

1. Given $A = [1, -3, 2]$, the function should return 2, as described above.
2. Given $A = [-3, 2, 4, -5, 3]$, the function should return 3. We can plant one tree in field number 0 and two trees in field number 4, achieving $[-2, 2, 4, -5, 5]$.
3. Given $A = [-2, 1, -3, 1]$, the function should return 4. We can plant two trees each in fields number 1 and 2. After that, we obtain values $[-2, 3, -1, 1]$.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array A is an integer within the range $[-10,000..10,000]$.

EndsTheSame

Initially, string S of length N is given. Then $N-1$ operations are applied to it: move the first letter of S to the end. How many times is the first letter of S the same as the last letter?

For example, given $S = \text{"abbaa"}$, the obtained sequence of strings is:

abbaa $\rightarrow$ **bbaaa** $\rightarrow$ **baaab** $\rightarrow$ **aaabb** $\rightarrow$ **aabba**

Three of them have the same first and last letter.

Write a function:

```
def solution(S)
```

that, given a string S of length N , consisting of letters 'a' and/or 'b', returns the number of times the first letter is the same as the last in the obtained sequence of strings.

Examples:

1. Given $S = \text{"abbaa"}$, the function should return 3, as described above.
2. Given $S = \text{"aaaa"}$, the function should return 4. The first and last letters are always the same.
3. Given $S = \text{"abab"}$, the function should return 0. The first and last letters are always different.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[2..200,000]$;
- string S is made only of the characters 'a' and/or 'b'.

AssassinsStealth

We are given a two-dimensional board of size $N \times M$ (N rows and M columns). Each field of the board can be empty ('.'), may contain an obstacle ('X') or may have a character in it. The character might be either an assassin ('A') or a guard. Each guard stands still and looks straight ahead, in the direction they are facing.

Every guard looks in one of four directions (up, down, left or right on the board) and is represented by one of four symbols. A guard denoted by '<' is looking to the left; by '>', to the right; '^', up; or 'v', down. The guards can see everything in a straight line in the direction in which they are facing, as far as the first obstacle ('X' or any other guard) or the edge of the board.

The assassin can move from the current field to any other empty field with a shared edge. The assassin cannot move onto fields containing obstacles or enemies.

Write a function:

```
def solution(B)
```

that, given an array B consisting of N strings denoting rows of the array, returns True if it is possible for the assassin to sneak from their current location to the bottom-right cell of the board undetected, and False otherwise.

Examples:

1. Given $B = ["X.....>", "..v..X.", ">..X..", "A....."]$, your function should return False. All available paths lead through a field observed by a guard.

X							>
		v			X		
	>			X			
A							

2. Given $B = ["...Xv", "AX..^", ".XX.."]$, your function should return True. The guard in the second row is blocking the other one from watching the bottom-right square.

			X	v
A	X			^
	X	X		

3. Given $B = ["...", ">.A"]$, your function should return False, as the assassin gets spotted right at the start.

>		A

4. Given $B = ["A.v", "..."]$, your function should return False. It's not possible for the assassin to enter the bottom-right cell undetected, as the cell is observed.

A		v

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..500]$;
- all strings in B are of the same length M from range $[1..500]$;
- there is exactly one assassin on the board;
- there is no guard or wall on $B[N-1][M-1]$;
- every string in B consists only of the following characters '!', 'X', '<', '>', 'v', '^' and/or 'A'.

FreeingStorageSpace

A storeroom is used to organize items stored in it on N shelves. Shelves are numbered from 0 to $N-1$. The K -th shelf is dedicated to items of only one type, denoted by a positive integer $A[K]$.

Recently it was decided that it is necessary to free R consecutive shelves. Shelves cannot be reordered. What is the maximum number of types of items which still can be stored in the storeroom after freeing R consecutive shelves?

Write a function:

```
def solution(A, R)
```

that, given an array A of N integers representing types of items stored on storeroom shelves, and an integer R representing the number of consecutive shelves to be freed, returns the maximum number of different types of items that can be stored in the storeroom after freeing R consecutive shelves.

Examples:

1. Given $A = [2, 1, 2, 3, 2, 2]$ and $R = 3$, your function should return 2. It can be achieved, for example, by freeing shelves 2, 3 and 4 (shelves are numbered from 0).
2. Given $A = [2, 3, 1, 1, 2]$ and $R = 2$, your function should return 3. All three types can still be stored by freeing the last two shelves.
3. Given $A = [20, 10, 10, 10, 30, 20]$ and $R = 3$, your function should return 3. It can be achieved by freeing the first three shelves.
4. Given $A = [1, 100000, 1]$ and $R = 3$, your function should return 0. All shelves need to be freed.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- R is an integer within the range $[1..N]$;
- each element of array A is an integer within the range $[1..100,000]$.

NoEqualDigits

You are given a positive integer N . Your task is to find the smallest integer greater than N that does not contain two identical consecutive digits.

For example, given $N = 1765$, the smallest integer greater than N is 1766. However, in 1766 the last two digits are identical. The next integer, 1767, does not contain two identical consecutive digits, and is the smallest integer greater than 1765 that fulfils the condition. Note that the second and fourth digits in 1767 can both be 7 as they are not consecutive.

Write a function:

```
def solution(N)
```

that, given an integer N , returns the smallest integer greater than N that does not contain two identical consecutive digits.

Examples:

1. Given $N = 55$, the function should return 56. It is the smallest integer greater than 55 and it does not contain two consecutive digits that are the same.
2. Given $N = 1765$, the function should return 1767, as explained above.
3. Given $N = 98$, the answer should be 101. Both 99 and 100 contain two identical consecutive digits, but 101 does not.
4. Given $N = 44432$, the answer should be 45010.
5. Given $N = 3298$, the answer should be 3401.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..1,000,000,000]$.

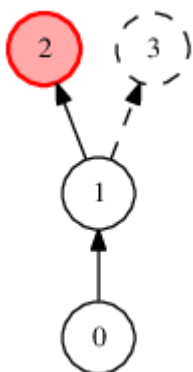
SkillTree

In role-playing video games a player assumes the role of a fictional character exploring the world, completing quests and learning new skills. The skill tree is one of the systems for progressing the character's development. It represents a hierarchy of skills.

The main difficulty with such system is that whenever the player wants to learn a particular new skill, there is another skill that has to be learned first. The exception to this rule is the easiest skill to learn in the game, as it has no prerequisites. The name "skill tree" derives from the fact that those dependencies create the structure of a tree, with the easiest skill being at the root.

Once a skill is learned, the character will always have it, so there is no need to learn it again if some other skill requires it. For example, if skill 1 requires skill 0, skill 2 requires skill 1 and skill 3 requires skill 1, then in order to learn skill 2, the player needs to learn three skills: 2, 1 and 0. If the player would also like to learn skill 3, then they need to learn only one more skill, skill 3, as its prerequisites, skills 1 and 0, have already been fulfilled.

In the pictures below, red color illustrates skills that are supposed to be learned, solid edges and vertices are ones that have to be learned to unlock red ones and dashed edges and vertices are ones that can be skipped.



You are given an array T of N integers that represents the skill tree. The K -th skill can be learned only if the $T[K]$ -th skill has already been learned. Skill 0 is the root and $T[0] = 0$. For the example explained above: $T[0] = 0$, $T[1] = 0$, $T[2] = 1$ and $T[3] = 1$, hence $T = [0, 0, 1, 1]$.

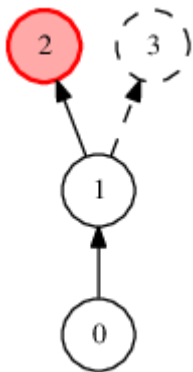
Write a function:

```
def solution(T, A)
```

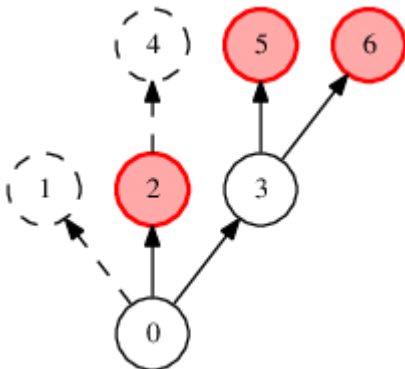
that, given an array T of N integers and an array A of M integers, returns the minimum number of skills which have to be learned to acquire all of the skills from the array A in the skill tree T.

Examples:

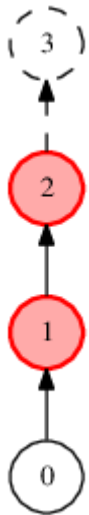
1. Given $T = [0, 0, 1, 1]$ and $A = [2]$, your function should return 3, as explained above.



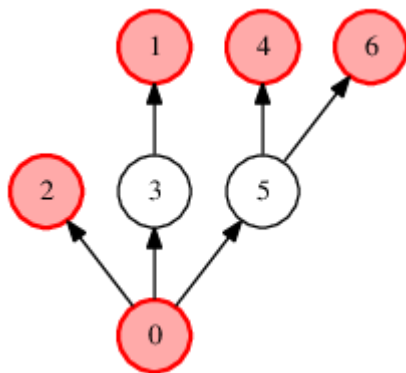
2. Given $T = [0, 0, 0, 0, 2, 3, 3]$ and $A = [2, 5, 6]$, your function should return 5. To learn skill numbers 2, 5, 6, skills $T[2] = 0$, $T[5] = 3$ and $T[6] = 3$ have to be learned. To learn skill number 3, skill $T[3] = 0$ has to be acquired. Skill 0 is the root. This gives five skills in total: 2, 5, 6, 3, 0.



3. Given $T = [0, 0, 1, 2]$ and $A = [1, 2]$, your function should return 3. Skill number 1 is necessary to unlock skill number 2, and skill number 1 requires skill number 0. That gives three skills in total.



4. Given $T = [0, 3, 0, 0, 5, 0, 5]$ and $A = [4, 2, 6, 1, 0]$, your function should return 7. Skills 4, 2, 6, 1 require respectively skills 5, 0, 5, 3. Skills 5 and 3 require skill 0. This gives seven different skills in total that have to be learned in order to acquire all skills from set A.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- M is an integer within the range $[1..N]$;
- each element of arrays T and A is an integer within the range $[0..N-1]$;
- skills form a tree rooted at 0;
- the elements of A are all distinct.

Note: The maximum depth of the Python interpreter stack (recursion limit) is set to 250,000 for this task.

RoadRoller

There are N potholes on a straight road parallel to the y -axis. The positions of the potholes are described by two arrays of integers, X and Y . The K -th pothole (for K within the range $[0..N-1]$) is located at coordinates $(X[K], Y[K])$.

In order to fix the potholes, a road roller of width W will be used. The road roller can only drive along the street (parallel to the y-axis). It can start driving from any x coordinate at the beginning of the road (a point whose y coordinate is equal to 0). During one drive, for a chosen starting point $(x, 0)$ of the road roller's left end, the road roller drives upwards and patches all potholes on its way that fall within the width of the road roller, W , and are to the right of its starting position, x . In other words, it patches all the potholes whose first coordinate is between x and $x+W$ inclusive.

What is the minimum number of road roller drives required to patch all the potholes?

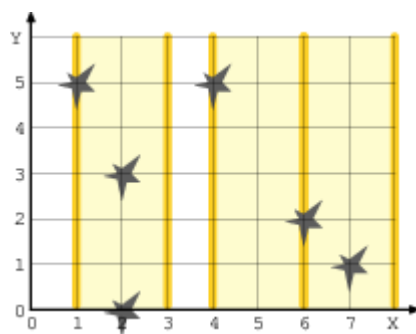
Write a function:

```
def solution(X, Y, W)
```

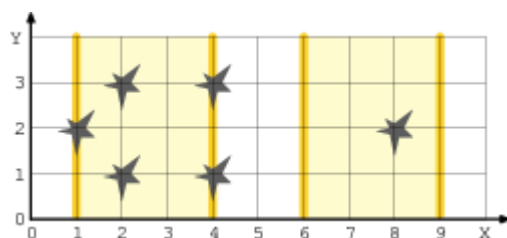
that, given two arrays of integers X and Y , describing the positions of the N potholes, and an integer W , specifying the width of the road roller, returns the minimum number of drives needed to patch all the potholes.

Examples:

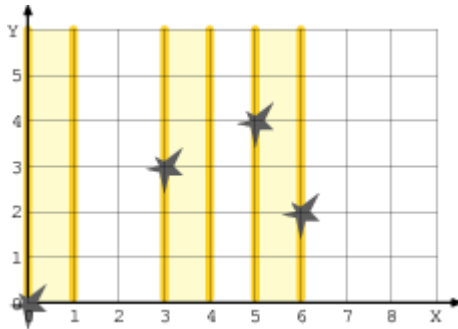
1. Given $X = [2, 4, 2, 6, 7, 1]$, $Y = [0, 5, 3, 2, 1, 5]$ and $W = 2$, the answer is 3. At least three drives will be needed to patch all the potholes. For example, the drives could start respectively at points $(1, 0)$, $(4, 0)$ and $(6, 0)$. During the first drive, the road roller would patch the potholes located at $(2, 0)$, $(2, 3)$ and $(1, 5)$. During the second drive, the potholes located at $(4, 5)$ and $(6, 2)$ would be patched. Finally, in the third drive, the pothole located at $(7, 1)$ would be patched.



2. Given $X = [4, 8, 2, 2, 1, 4]$, $Y = [1, 2, 3, 1, 2, 3]$ and $W = 3$, the answer is 2. If the first drive of the road roller started at point $(1, 0)$ and the second drive at the point $(6, 0)$, all the potholes would be patched.



3. Given $X = [0, 3, 6, 5]$, $Y = [0, 3, 2, 4]$ and $W = 1$, the answer is 3. The first drive of the road roller could start at point (0, 0), the second drive at point (3, 0) and the third at point (5, 0).



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of arrays X and Y is an integer within the range $[0..1,000,000,000]$;
- W is an integer within the range $[1..1,000,000,000]$.

RoadRepair

You are given a task to fix potholes in a road. The road is described by a string S consisting of N characters. Each character represents a single fragment of the road. Character '.' denotes a smooth surface and 'x' denotes a pothole. For example, $S = "...xxx..x"$ means that the road starts with three smooth fragments, followed by three potholes, followed by two smooth fragments and ending with one pothole.

You can choose any number of consecutive potholes and fix all of them. Fixing a segment consisting of K consecutive potholes costs $K + 1$. In the example above, fixing the first two consecutive potholes costs $2 + 1 = 3$ and fixing the last pothole costs $1 + 1 = 2$. After these fixes, the road would look like this: `".....x..."`.

You are given a budget B . You can fix multiple segments containing potholes as long as you fit in the budget. What is the maximum number of potholes you can fix?

Write a function:

```
def solution(S, B)
```

that, given the string S of length N and the integer B , returns the maximum number of potholes that can be fixed.

Examples:

1. Given $S = "...xxx..x....xxx."$ and $B = 7$, the function should return 5. You can start by fixing the first three consecutive potholes for a cost of 4, obtaining the road: $".....x....xxx."$. Then, you can fix the last two potholes for a cost of 3, obtaining the road: $".....x....x..."$. The total cost is 7, which fits in the budget, and you fix 5 potholes in total.

2. Given $S = "..xxxxx"$ and $B = 4$, the function should return 3. One way is to fix the middle three potholes, which costs the whole budget and makes the road look as follows: $"..x...x"$. Alternatively, you could fix the first three potholes or the last three potholes.

3. Given $S = "x.x.xxx...x"$ and $B = 14$, the function should return 6. You can fix all the potholes, which costs $2 + 2 + 4 + 2 = 10$, leaving you with the spare budget of 4. This fixes the entire road.

4. Given $S = ".."$ and $B = 5$, the function should return 0. There are no potholes to fix.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- B is an integer within the range $[0..200,000]$;
- string S consists only of characters $'.'$ and $'x'$.

GardenArrangement

A garden is divided into N sections numbered from 0 to $N-1$. It is described by an array A , where $A[K]$ denotes the number of trees in the K -th section. To make the garden look more organized, we want the number of trees in every section to be the same. As we don't want to cut any trees down, we can perform either of the following actions:

- planting a new tree in one of the sections;
- replanting an existing tree, moving it from one section to another.

We want to minimize the number of actions performed.

Write a function:

```
def solution(A)
```

that, given an array A consisting of N integers describing the garden, returns the minimum number of actions we need to perform in order to make all sections of the garden contain the same number of trees.

Examples:

1. Given $A = [1, 2, 2, 4]$ the function should return 4. We can move one tree from $A[3]$ to $A[1]$ and obtain $A = [1, 3, 2, 3]$. Then we can plant two trees in $A[0]$ and one tree in $A[2]$ to make every section contain three trees.

2. Given $A = [4, 2, 4, 6]$, the function should return 2. We can move two trees from $A[3]$ to $A[1]$. This way, every section in the garden will contain four trees.

3. Given $A = [1, 1, 2, 1]$, the function should return 3. We can plant one tree in $A[0]$, $A[1]$ and $A[3]$ so that each section in the garden contains two trees.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array A is an integer within the range $[1..1,000,000,000]$;
- the answer is always less than or equal to $2,000,000,000$.

TwoWayRoadRenovation

You are given a description of a two-lane road in which two strings, $L1$ and $L2$, respectively represent the first and the second lane, each lane consisting of N segments of equal length.

The K -th segment of the first lane is represented by $L1[K]$ and the K -th segment of the second lane is represented by $L2[K]$, where '.' denotes a smooth segment of road and 'x' denotes a segment containing potholes.

Cars can drive over segments with potholes, but it is rather uncomfortable. Therefore, a project to repair as many potholes as possible was submitted. At most one contiguous stretch of each lane may be repaired at a time. For the time of reparation those stretches will be closed to traffic.

How many road segments with potholes can be repaired given that the road must be kept open (in other words, stretches of roadworks must not prevent travel from one end of the road to the other)?

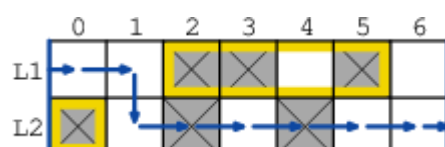
Write a function:

```
def solution(L1, L2)
```

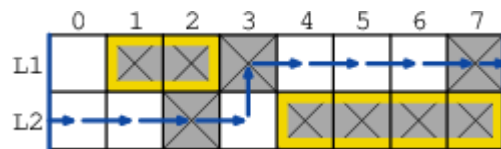
that, given two strings $L1$ and $L2$ of length N , returns the maximum number of segments with potholes that can be repaired.

Examples:

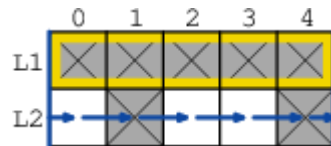
1. Given $L1 = "..xx.x."$ and $L2 = "x.x.x.."$, your function should return 4. It is possible to repair three potholes in the first lane and the first pothole in the second lane without closing the road to traffic.



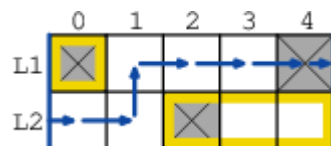
2. Given L1 = ".xxx...x" and L2 = "..x.xxxx", your function should return 6.



3. Given L1 = "xxxxx" and L2 = ".x..x", your function should return 5.



4. Given L1 = "x...x" and L2 = "..x..", your function should return 2.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..200,000];
- strings L1 and L2 consist only of the characters "." and/or "x".

RecyclingTrucks

There are N houses (numbered from 0 to N-1) along a street. In each of them, recyclable trash (plastic, glass, metal) is collected into separate bags.

There are three trucks that collect the trash. Each of them collects a separate type of trash (the first collects plastic, the second, glass and the third, metal). All the trucks begin and end their jobs at the starting end of the street. Passing from the starting point to house number 0 takes D[0] minutes. Passing between houses number K-1 and K (for K in the range 1 to N-1) takes D[K] minutes. Loading one bag onto the truck takes one minute.

For example, D = [2, 5] means that passing between the starting point and house number 0 takes 2 minutes and passing between houses number 0 and 1 takes 5 minutes.

Each of the houses has already collected some bags (or possibly no bags) of recyclable trash. The number of bags that house number K has collected is recorded in string T[K], composed of letters 'P' (plastic), 'G' (glass) and 'M' (metal). For example, T[1] = "GMG" means that house number 1 has collected two bags of glass and one bag of metal. Each house may collect more than one bag of each type.

All of the trucks start their jobs simultaneously. Each finishes its job after collecting all of the bags of the given type of trash and returning back to the starting point. What is the minimum number of minutes that will pass before all the trucks finish all the jobs?

Write a function:

```
def solution(D, T)
```

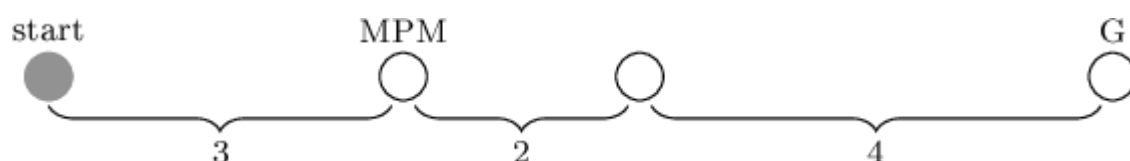
that, given array D of N integers and array T of N strings, returns the minimum number of minutes needed by the trucks to finish all the jobs.

Examples:

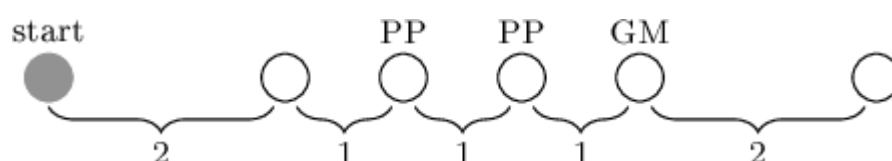
1. Given $D = [2, 5]$, $T = ["PGP", "M"]$, the function should return 15. The truck collecting plastic needs to go to house number 0, collect two bags and go back, which takes $2 + 1 + 1 + 2 = 6$ minutes. The truck collecting glass needs 5 minutes: 2 minutes to go to house number 0, 1 minute to collect a bag and 2 minutes to return to the starting point. The truck collecting metal will go straight to house number 1, collect a bag and go back in $7 + 1 + 7 = 15$ minutes. After 15 minutes all of the trash will have been collected and all trucks will have returned to the starting point.



2. Given $D = [3, 2, 4]$, $T = ["MPM", "", "G"]$, the function should return 19. The truck collecting glass needs the most time: $3 + 2 + 4 + 1 + 4 + 2 + 3 = 19$.



3. Given $D = [2, 1, 1, 1, 2]$, $T = ["", "PP", "PP", "GM", ""]$, the function should return 12. The truck collecting plastic needs 12 minutes, whereas the trucks collecting glass and metal both need 11 minutes to finish their jobs.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array D is an integer within the range $[1..100]$;
- each element of array T is a string consisting of letters 'P', 'G' or 'M';

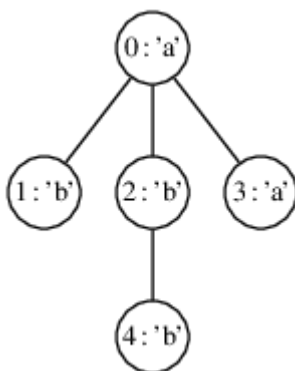
- S , the total length of all strings, is an integer within the range $[0..100,000]$.

VaryingPathInTree

You are given a tree consisting of N nodes, numbered from 0 to $N-1$. Each node contains one of the letters 'a' or 'b'.

The tree is represented as an array A of length N . $A[K]$ (for K from 0 to $N-1$) denotes the parent of the K -th node. Node 0 is the root node and does not have a parent, so the value corresponding to it in array A will always be -1. Letters in nodes are represented as a string S . $S[K]$ (for K from 0 to $N-1$) denotes the letter in the K -th node.

For example, $S = \text{"abbab"}$ and $A = [-1, 0, 0, 0, 2]$ represents a tree made of five vertices. It contains following edges: $0 - 1$, $0 - 2$, $0 - 3$, $2 - 4$. The picture below illustrates the tree.



Your task is to find the number of vertices on the longest path in the tree, such that no pair of adjacent vertices on the path would contain the same letter.

Write a function:

```
def solution(S, A)
```

that, given string S and array A of N integers, returns an integer representing the number of vertices on the longest path in which no two consecutive vertices contain the same letter.

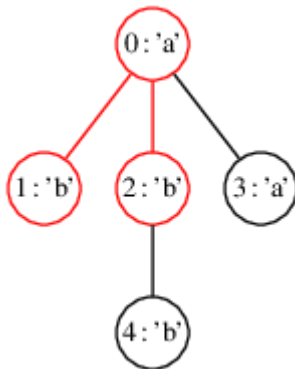
Examples:

1. Given $S = \text{"ab"}$ and $A = [-1, 0]$, your function should return 2. The longest path is: $0 - 1$. It is marked in red on the picture.



2. Given $S = \text{"abbab"}$ and $A = [-1, 0, 0, 0, 2]$, your function should return

3. The longest path is: $1 - 0 - 2$. It is marked in red on the picture.



3. Given $S = \text{"abab"}$ and $A = [-1, 2, 0, 1]$, your function should return 2.

The longest path is: $2 - 1$. It is marked in red on the picture.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..40,000]$;
- string S is made only of the characters 'a' and/or 'b';
- string S and array A are both of length N ;
- array A defines a proper tree with a root in node 0.

Note: The maximum depth of the Python interpreter stack (recursion limit) is set to 80,000 for this task.

DivisibleByThree

You are given a string *S*, consisting of *N* digits, that represents a number. You can change at most one digit in the string to any other digit. How many different numbers divisible by 3 can be obtained in this way?

Write a function:

```
def solution(S)
```

that, given a string *S* of length *N*, returns an integer specifying how many numbers divisible by 3 can be obtained with at most one change of a digit.

Examples:

1. Given *S* = "23", the function should return 7. All numbers divisible by 3 that can be obtained after at most one change are: "03", "21", "24", "27", "33", "63" and "93".
2. Given *S* = "0081", the function should return 11. All numbers divisible by 3 that can be obtained with at most one change are: "0021", "0051", "0081", "0084", "0087", "0381", "0681", "0981", "3081", "6081" and "9081".
3. Given *S* = "022", the function should return 9. All numbers divisible by 3 that can be obtained with at most one change are: "012", "021", "024", "027", "042", "072", "222", "522" and "822".

Write an efficient algorithm for the following assumptions:

- *N* is an integer within the range [1..100,000];
- string *S* is made only of digits (0-9).

Remember, all submissions are being checked for plagiarism. Your recruiter will be informed in case suspicious activity is detected.

PathDetection

You are given an undirected graph consisting of *N* vertices, numbered from 1 to *N*, and *M* edges.

The graph is described by two arrays, *A* and *B*, both of length *M*. A pair (*A*[*K*], *B*[*K*]), for *K* from 0 to *M*-1, describes an edge between vertex *A*[*K*] and vertex *B*[*K*].

Your task is to check whether the given graph contains a path from vertex 1 to vertex *N* going through all of the vertices, one by one, in increasing order of their numbers. All connections on the path should be direct.

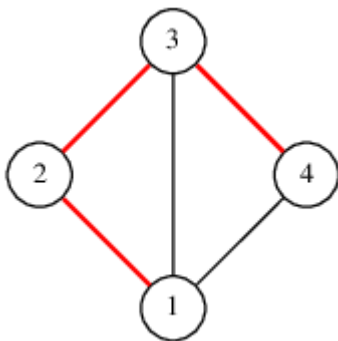
Write a function:

```
def solution(N, A, B)
```

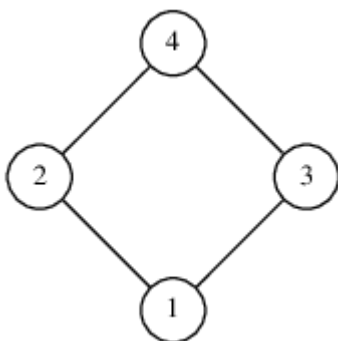
that, given an integer N and two arrays A and B of M integers each, returns `True` if there exists a path from vertex 1 to N going through all vertices, one by one, in increasing order, or `False` otherwise.

Examples:

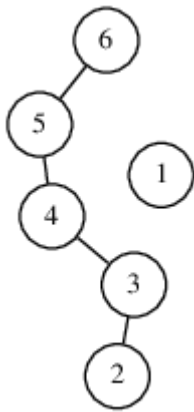
1. Given $N = 4$, $A = [1, 2, 4, 4, 3]$ and $B = [2, 3, 1, 3, 1]$, the function should return `True`. There is a path $(1 \rightarrow 2 \rightarrow 3 \rightarrow 4)$ using edges $(1, 2)$, $(2, 3)$ and $(4, 3)$.



2. Given $N = 4$, $A = [1, 2, 1, 3]$ and $B = [2, 4, 3, 4]$, the function should return `False`. There is no path $(1 \rightarrow 2 \rightarrow 3 \rightarrow 4)$, as there is no direct connection from vertex 2 to vertex 3.



3. Given $N = 6$, $A = [2, 4, 5, 3]$ and $B = [3, 5, 6, 4]$, the function should return `False`. There is no direct connection from vertex 1 to vertex 2.



4. Given $N = 3$, $A = [1, 3]$ and $B = [2, 2]$, the function should return `True`. There is a path $(1 \rightarrow 2 \rightarrow 3)$ using edges $(1, 2)$ and $(3, 2)$.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[2..100,000]$;
- M is an integer within the range $[0..100,000]$;
- all elements of arrays A and B are integers within the range $[1..N]$;
- there are no self-loops (edges with $A[K] = B[K]$) in the graph;
- there are no multiple edges between the same vertices.

EqualSegments

You are given an array A of integers. Find the maximum number of non-intersecting segments of length 2 (two adjacent elements), such that segments have an equal sum.

For example, given $A = [10, 1, 3, 1, 2, 2, 1, 0, 4]$, there are three non-intersecting segments, each whose sum is equal to 4: $(1, 3)$, $(2, 2)$, $(0, 4)$. Another three non-intersecting segments are: $(3, 1)$, $(2, 2)$, $(0, 4)$.

Write a function:

```
def solution(A)
```

that, given an array A of N integers, returns the maximum number of segments with equal sums.

Examples:

1. Given $A = [10, 1, 3, 1, 2, 2, 1, 0, 4]$, the function should return 3, as explained above.
2. Given $A = [5, 3, 1, 3, 2, 3]$, the function should return 1. Each sum of two adjacent elements is different from the others.

3. Given $A = [9, 9, 9, 9, 9]$, the function should return 2.

4. Given $A = [1, 5, 2, 4, 3, 3]$, the function should return 3. There are three segments: (1, 5), (2, 4), (3, 3) whose sums are equal to 6.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[2..100,000]$;
- each element of array A is an integer within the range $[0..1,000,000,000]$.

MinDistinct

You are given an array A consisting of N integers within the range $[1..N]$. In one move, you can increase or decrease the value of any element by 1. After each move, all numbers should remain within the range $[1..N]$.

Your task is to find the smallest required number of moves to make all elements in the array pairwise distinct (in other words, no value can appear in the array more than once).

Write a function:

```
int solution(int A[], int N);
```

that, given an array A consisting of N integers, returns the smallest number of moves required to make all elements in the array pairwise distinct. If the result is greater than 1,000,000,000, the function should return -1.

Examples:

1. Given $A = [1, 2, 1]$, the function should return 2, because you can increase $A[2]$ twice: $[1, 2, 1] \rightarrow [1, 2, 2] \rightarrow [1, 2, 3]$. In this example, you could also change the array to the following values in two moves: $[3, 2, 1]$, $[1, 3, 2]$, $[2, 3, 1]$.

2. Given $A = [2, 1, 4, 4]$, the function should return 1, as it is sufficient to decrease $A[2]$ or $A[3]$ by 1, resulting in $[2, 1, 3, 4]$ or $[2, 1, 4, 3]$.

3. Given $A = [6, 2, 3, 5, 6, 3]$, the function should return 4, because you can achieve the following array in four moves: $[6, 2, 1, 5, 4, 3]$.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..200,000]$;
- each element of array A is an integer within the range $[1..N]$.

LongestEvenCount

Write a function:

```
def solution(S)
```

that, given a string *S* consisting of *N* lowercase English letters, returns the length of the longest substring in which every letter occurs an even number of times. A substring is defined as a contiguous segment of a string. If no such substring exists, return 0.

Examples:

1. Given *S* = "bdaaadadb", the function should return 6. Substrings in which every letter occurs an even number of times are "aa", "adad", "daaada" and "aaadad". The length of the longest of them is 6.
2. Given *S* = "abacb", the function should return 0. There is no non-empty substring in which every letter occurs an even number of times.
3. Given *S* = "zthtzh", the function should return 6. Every letter in the whole string occurs an even number of times.

Write an efficient algorithm for the following assumptions:

- *N* is an integer within the range [1..100,000];
- string *S* consists only of lowercase letters ('a'-'z').

LargestPalindrome

You are given a string *S*, which consists entirely of decimal digits (0-9). Using digits from *S*, create a palindromic number with the largest possible decimal value. You should use at least one digit and you can reorder the digits. A palindromic number remains the same when its digits are reversed; for instance, "7", "44" or "83238". Any palindromic number you create should not, however, have any leading zeros, such as in "0990" or "010".

For example, decimal palindromic numbers that can be created from "8199" are: "1", "8", "9", "99", "919" and "989". Among them, "989" has the largest value.

Write a function:

```
def solution(S)
```

that, given a string S of N digits, returns the string representing the palindromic number with the largest value.

Examples:

1. Given "39878", your function should return "898".
2. Given "00900", your function should return "9".
3. Given "0000", your function should return "0".
4. Given "54321", your function should return "5".

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- string S is made only of digits (0-9).

DeleteDefectiveRows

You are given a table containing data. The first row contains the column headers; all the other rows contain the data. Some of the rows are defective. A row is defective if it contains at least one cell with a NULL value written in it. You are supposed to delete all of the defective rows.

In the sample below, the second row is defective. It contains a NULL in the age column. The first row (the header) is never defective. Every cell contains a single word. Each word may contain only digits ('0' - '9'), lowercase and uppercase English letters ('a' - 'z', 'A' - 'Z'); for example:

```
+----+-----+-----+-----+
| id |  name |  age |  score |
+----+-----+-----+-----+
|  1 |  Jack | NULL |    12 |
+----+-----+-----+-----+
| 17 | Betty |   28 |    11 |
+----+-----+-----+-----+
```

For instance, in the example above, after removing the defective row the table should look like this:

```
+----+-----+-----+-----+
| id |  name |  age |  score |
+----+-----+-----+-----+
| 17 | Betty |   28 |    11 |
+----+-----+-----+-----+
```

You cannot change the order of rows. The number of rows and columns may differ in different test cases. The case of the letters matters ('a' ≠ 'A').

The table is given in CSV (comma-separated values) format. Every two consecutive cells in each row are separated by a single comma ',' symbol. Every two consecutive rows are separated by a new-line '\n' symbol. For example, the first table from the task statement, written in CSV format, is the single string: "id,name,age,score\n1,Jack,NULL,12\n17,Betty,28,11". You may assume that each row has the same number of cells.

Write a function:

```
def solution(S)
```

which, given a string S of length N, returns the table without the defective rows in a CSV format.

Examples:

1. Given S = "id,name,age,score\n1,Jack,NULL,12\n17,Betty,28,11", your function should return "id,name,age,score\n17,Betty,28,11".

The table with data from string S looks as follows:

```
+----+-----+-----+-----+
| id |  name |  age |  score |
+----+-----+-----+-----+
|  1 |  Jack | NULL |    12 |
+----+-----+-----+-----+
| 17 | Betty |   28 |    11 |
+----+-----+-----+-----+
```

After removing the defective rows, the table should look like this:

```
+----+-----+-----+-----+
| id |  name |  age |  score |
+----+-----+-----+-----+
| 17 | Betty |   28 |    11 |
+----+-----+-----+-----+
```

2. Given S = "header,header\nANNUL,ANNULLED\nnull,NILL\nNULL,NULL", your function should return "header,header\nANNUL,ANNULLED\nnull,NILL".

The table with data from string S looks as follows:

```
+-----+-----+
| header |  header |
+-----+-----+
|  ANNUL | ANNULLED |
```

```

+-----+-----+
| null | NILL |
+-----+-----+
| NULL | NULL |
+-----+-----+

```

After removing the defective rows, the table should look like this:

```

+-----+-----+
| header | header |
+-----+-----+
| ANNUL | ANNULLED |
+-----+-----+
| null | NILL |
+-----+-----+

```

The second row is not defective, because the "null" is written in lowercase.

3. Given `S = "country,population,area\nUK,67m,242000km2"`, your function should return `"country,population,area\nUK,67m,242000km2"`.

The table with data from string `S` looks as follows:

```

+-----+-----+-----+
| country | population | area |
+-----+-----+-----+
| UK | 67m | 242000km2 |
+-----+-----+-----+

```

There are no defective rows in this table, hence after removing defective rows the table looks the same.

Write an efficient algorithm for the following assumptions:

- `N` is an integer within the range `[1..200,000]`;
- `S` is a string of length `N` in CSV format;
- there is at least one row;
- each row has the same, positive number of cells;
- each cell consists only of letters and/or digits;
- the first row does not contain a `NULL` cell.

AngryFrogs

There are N blocks, numbered from 0 to $N-1$, arranged in a row. A couple of frogs were sitting together on one block when they had a terrible quarrel.

Now they want to jump away from one another so that the distance between them will be as large as possible. The distance between blocks numbered J and K , where $J \leq K$, is computed as $K - J + 1$. The frogs can only jump up, meaning that they can move from one block to another only if the two blocks are adjacent and the second block is of the same or greater height as the first. What is the longest distance that they can possibly create between each other, if they also chose to sit on the optimal starting block initially?

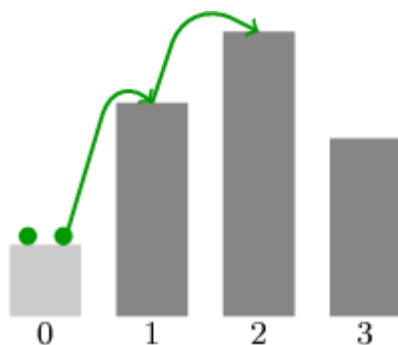
Write a function:

```
def solution(blocks)
```

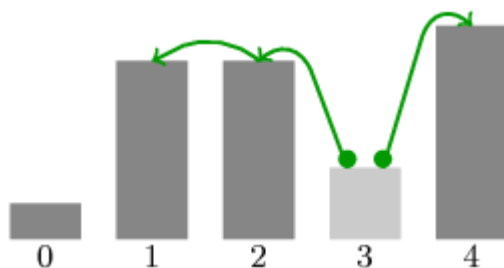
that, given an array `blocks` consisting of N integers denoting the heights of the blocks, returns the longest possible distance that two frogs can make between each other starting from one of the blocks.

Examples:

1. Given `blocks = [2, 6, 8, 5]`, the function should return 3. If starting from `blocks[0]`, the first frog can stay where it is and the second frog can jump to `blocks[2]` (but not to `blocks[3]`).



2. Given `blocks = [1, 5, 5, 2, 6]`, the function should return 4. If starting from `blocks[3]`, the first frog can jump to `blocks[1]`, but not `blocks[0]`, and the second frog can jump to `blocks[4]`.



3. Given `blocks = [1, 1]`, the function should return 2. If starting from `blocks[1]`, the first frog can jump to `blocks[0]` and the second frog can stay where it is. Starting from `blocks[0]` would result in the same distance.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range [2..200,000];
- each element of array blocks is an integer within the range [1..1,000,000,000].

TwoRooks

You are given a matrix A representing a chessboard with N rows and M columns. Each square of the chessboard contains an integer representing a points-based score. You have to place two rooks on the chessboard in such a way that they cannot attack each other and the sum of points on their squares is maximal. Rooks in chess can attack each other only if they are in the same row or column.

For example, given a matrix as in the following picture:

	0	1
0	1	4
1	2	3

we can place rooks in two different ways:

- One rook on $A[0][0] = 1$ and another rook on $A[1][1] = 3$. The sum of points on these squares is 4.
- One rook on $A[0][1] = 4$ and another rook on $A[1][0] = 2$. The sum of points on these squares is 6.

Your task is to find the maximum sum of two squares of the chessboards on which the rooks can be placed. In the example above, the answer is 6. We cannot, for example, place the rooks at $A[0][1]$ and $A[1][1]$ (whose sum is 7), as they would attack each other.

Write a function:

```
def solution(A)
```

which, given a matrix A, returns the maximum sum that can be achieved by placing two non-attacking rooks.

Examples:

1. Given matrix A with two rows and two columns:

	0	1
0	1	4
1	2	3

the function should return 6. We can achieve the maximum sum by selecting $A[0][1] + A[1][0] = 4 + 2$. The selected squares are marked in green.

2. Given matrix A with three rows and three columns:

	0	1	2
0	15	1	5
1	16	3	8
2	2	6	4

the function should return 23. We can achieve the maximum sum by selecting $A[0][0] + A[1][2] = 15 + 8$. The selected squares are marked in green.

3. Given matrix A with three rows and two columns:

	0	1
0	12	12
1	12	12
2	0	7

the function should return 24. We can achieve the maximum sum by selecting $A[0][0] + A[1][1] = 12 + 12$ or $A[0][1] + A[1][0] = 12 + 12$. The latter solution is marked in green.

4. Given matrix A with two rows and three columns:

	0	1	2
0	1	2	14
1	8	3	15

the function should return 22. We can achieve the maximum sum by selecting $A[0][2] + A[1][0] = 14 + 8$. The selected squares are marked in green.

Write an efficient algorithm for the following assumptions:

- N and M are integers within the range [2..600];

- each element of matrix A is an integer within the range [0..1,000,000,000].

CoverMaximalPoints

You are given an array A of N distinct integers and a positive integer K. Values of A represent points on an axis. What is the maximal number of points that can be covered using one interval of length K?

For example, given array A = [45, 1, 9, 13, 26, 12, 29, 7] and K = 5, we can cover at most three points. Some possible sets of points that can be covered by an interval of length 5 are listed below:

- points = [7, 9, 12]: the difference between the rightmost and leftmost point is equal to $12 - 7 = 5$, which is equal to K.
- points = [9, 12, 13]: the difference between the rightmost and leftmost point is equal to $13 - 9 = 4$, which is less than K.

Write a function:

```
def solution(A, K)
```

that, given an array A of N distinct integers and a positive integer K, returns the maximal number of points that can be covered by an interval of length K.

Examples:

1. Given A = [45, 1, 12, 16, 26, 15, 29, 7] and K = 5, the function should return 3, as explained above.
2. Given A = [4, 2, 1, 3] and K = 3, the function should return 4. All points can be covered using one interval. The difference between the rightmost and leftmost point is equal to $4 - 1 = 3$.
3. Given A = [11, 7, 1, 4] and K = 2, the function should return 1. Only one point can be covered at once using an interval of length K = 2. Distances between the points are larger than K.
4. Given A = [] and K = 11, the function should return 0. There are no points to cover.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [0..100,000];
- each element of array A is an integer within the range [0..1,000,000,000];

- array elements are distinct;
- K is an integer within the range [1..1,000,000,000].

Sticks

There are two wooden sticks of lengths A and B respectively. Each of them can be cut into shorter sticks of integer lengths. Our goal is to construct the largest possible square. In order to do this, we want to cut the sticks in such a way as to achieve four sticks of the same length (note that there can be some leftover pieces). What is the longest side of square that we can achieve?

Write a function:

```
def solution(A, B)
```

that, given two integers A, B, returns the side length of the largest square that we can obtain. If it is not possible to create any square, the function should return 0.

Examples:

1. Given A = 10, B = 21, the function should return 7. We can split the second stick into three sticks of length 7 and shorten the first stick by 3.
2. Given A = 13, B = 11, the function should return 5. We can cut two sticks of length 5 from each of the given sticks.
3. Given A = 2, B = 1, the function should return 0. It is not possible to make any square from the given sticks.
4. Given A = 1, B = 8, the function should return 2. We can cut stick B into four parts.

Write an efficient algorithm for the following assumptions:

- A and B are integers within the range [1..1,000,000,000].

FairSplits

You are given two arrays A and B consisting of N integers each.

Index K is named *fair* if the four sums $(A[0] + \dots + A[K-1])$, $(A[K] + \dots + A[N-1])$, $(B[0] + \dots + B[K-1])$ and $(B[K] + \dots + B[N-1])$ are all equal. In other words, K is the index where the two arrays, A and B, can be split (into two non-empty arrays each) in such a way that the sums of the resulting arrays' elements are equal.

For example, given arrays A = [0, 4, -1, 0, 3] and B = [0, -2, 5, 0, 3], index K = 3 is *fair*.

	0	1	2	3	4
A	0	4	-1	0	3
B	0	-2	5	0	3

K=3

The sums of the four subarrays are all equal: $0 + 4 + (-1) = 3$; $0 + 3 = 3$; $0 + (-2) + 5 = 3$ and $0 + 3 = 3$. On the other hand, index $K = 2$ is not *fair*; the sums of the subarrays are: $0 + 4 = 4$; $(-1) + 0 + 3 = 2$; $0 + (-2) = -2$ and $5 + 0 + 3 = 8$.

Write a function:

```
def solution(A, B)
```

which, given two arrays of integers A and B, returns the total number of *fair* indexes.

Examples:

1. Given $A = [0, 4, -1, 0, 3]$ and $B = [0, -2, 5, 0, 3]$, your function should return 2. The *fair* indexes are 3 and 4. In both cases, the sums of elements of the subarrays are equal to 3.
2. Given $A = [2, -2, -3, 3]$ and $B = [0, 0, 4, -4]$, your function should return 1. The only *fair* index is 2. Index 4 is not *fair* as the subarrays containing indexes from K to N - 1 would be empty.
3. Given $A = [4, -1, 0, 3]$ and $B = [-2, 6, 0, 4]$, your function should return 0. There are no *fair* indexes.
4. Given $A = [3, 2, 6]$ and $B = [4, 1, 6]$, your function should return 0.
5. Given $A = [1, 4, 2, -2, 5]$, $B = [7, -2, -2, 2, 5]$, your function should return 2. The *fair* indexes are 2 and 4.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[2..100,000]$;
- each element of arrays A and B is an integer within the range $[-1,000,000,000..1,000,000,000]$.

SquareTiles

You have M square tiles of size 1×1 and N square tiles of size 2×2 . Your task is to create the largest possible square using these tiles. Tiles may not overlap, and the resulting square should be filled (it should not contain empty spaces).

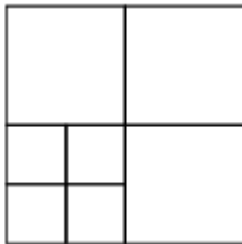
Write a function:

```
def solution(M, N)
```

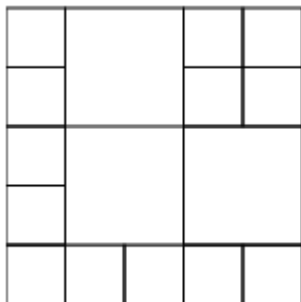
that, given two integers M and N, returns the length of the side of the largest square you can create. If no square can be created, your function should return 0.

Examples:

1. Given M = 8 and N = 0, your function should return 2. You can use four out of eight tiles to arrange them into 2×2 square. There are not enough tiles to create 3×3 square.
2. Given M = 4 and N = 3, your function should return 4. You can obtain a 4×4 square by arranging four 1×1 tiles into a 2×2 square, and surrounding it by 2×2 tiles:



3. Given M = 0 and N = 18, your function should return 8. You need to use sixteen 2×2 tiles to create the square. Note that not all the tiles are used.
4. Given M = 13 and N = 3, your function should return 5. One of the possible arrangements is shown in the following image:



Write an efficient algorithm for the following assumptions:

- M and N are integers within the range [0..1,000,000,000].

WordSplit

You are given a string consisting of lowercase letters of the English alphabet. You must split this string into a minimal number of substrings in such a way that no letter occurs more than once in each substring.

For example, here are some correct splits of the string "abacdec": ('a', 'bac', 'dec'), ('a', 'bacd', 'ec') and ('ab', 'ac', 'dec').

Write a function:

```
def solution(S)
```

that, given a string S of length N, returns the minimum number of substrings into which the string has to be split.

Examples:

1. Given 'world', your function should return 1. There is no need to split the string into substrings as all letters occur just once.
2. Given 'dddd', your function should return 4. The result can be achieved by splitting the string into four substrings ('d', 'd', 'd', 'd').
3. Given 'cycle', your function should return 2. The result can be achieved by splitting the string into two substrings ('cy', 'cle') or ('c', 'ycle').
4. Given 'abba', your function should return 2. The result can be achieved by splitting the string into two substrings ('ab', 'ba').

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..1,000,000];
- string S is made only of lowercase letters (a-z).

ReverseEdges

There is a network with N roads and N + 1 cities. The cities are labeled with distinct integers within the range [0..N].

Roads connect the cities in such a way that there is exactly one way to travel between any two of the cities. In other words, the network forms a *tree*.

The roads in the network are too narrow to accommodate two cars. For this reason, every road (that connects cities A and B) is oriented in one of two possible ways: either from A to B, or from B to A. If the road is oriented from A to B, then every car traveling from B to A has to give way to cars traveling from A to B. This, naturally, makes traveling from A to B much faster than traveling from B to A.

A big hospital was recently founded in the city labeled 0. For that reason, the citizens have decided to rearrange the orientation of the roads so that everyone can get to the hospital as

quickly as possible. This means that the trip from every city to the 0th city should not go through any road that faces against the current direction of travel. In order to minimize the cost of this operation, they would like to reorient as few roads as possible.

Write a function:

```
def solution(A, B)
```

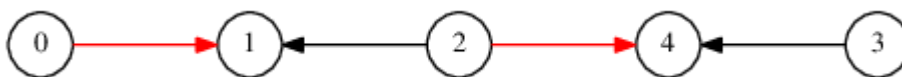
that, given the description of the network as two arrays A, B of N integers each, returns the minimum number of roads that must be reoriented in order to make everyone's trip to the hospital as fast as possible. Arrays A and B describe the network in the following way: for each K in the range [0..N-1], there is a road between cities labeled A[K] and B[K] that is oriented from A[K] to B[K].

Examples:

1. Given A = [0, 2, 2, 3] and B = [1, 1, 4, 4], your function should return 2.

The citizens should reorient the roads between the following cities:

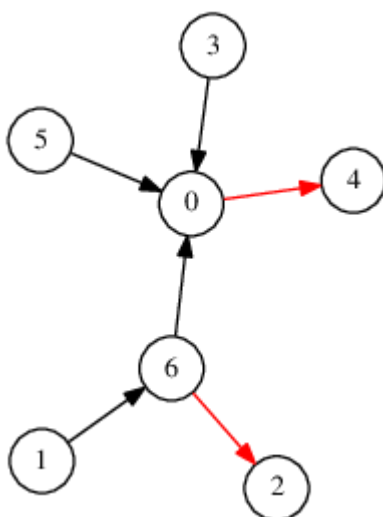
- 0 and 1,
- 2 and 4.



(roads to be reversed are marked in red).

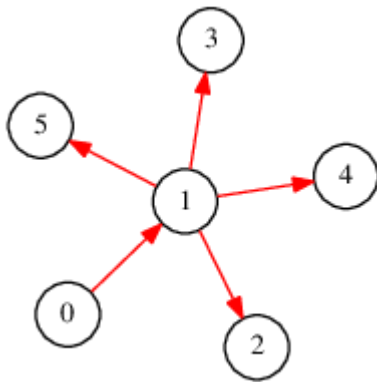
2. Given A = [1, 6, 6, 3, 0, 5] and B = [6, 2, 0, 0, 4, 0], your function should return 2. The citizens should reorient the roads between the following cities:

- 0 and 4,
- 6 and 2.



3. Given A = [0, 1, 1, 1, 1] and B = [1, 2, 3, 4, 5], your function should return 5.

The citizens should reorient all the roads:



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- arrays A and B contain integers from range $[0..N]$ and every such integer appears at least once in A or in B ;
- arrays A and B have equal lengths;
- there is exactly one way to travel between every pair of cities.

EqualFractions

You are given N fractions, numbered from 0 to $N-1$. Your task is to count the number of occurrences of the fraction that appears most often.

Notice that two fractions may be equal even if they have different representations; for example, $3/5$ and $6/10$ are equal.

You should not assume that floating point operations are precise.

Write a function:

```
def solution(X, Y)
```

that, given two arrays X and Y consisting of N integers each (which represent the K -th fraction as $X[K]/Y[K]$), returns the number of occurrences of the fraction that appears most often.

Examples:

1. Given arrays $X = [1, 2, 3, 4, 0]$, $Y = [2, 3, 6, 8, 4]$, the function should return 3, as fractions $1/2$, $3/6$, $4/8$ are all equal to each other.
2. Given arrays $X = [3, 3, 4]$, $Y = [5, 4, 3]$, the function should return 1, as no two fractions are equal.
3. Given arrays $X = [4, 4, 7, 1, 2]$, $Y = [4, 4, 8, 1, 2]$, the function should return 4, as fractions $4/4$, $4/4$, $1/1$, $2/2$ are all equal to each other.

4. Given arrays $X = [1, 2, 3, 1, 2]$, $Y = [2, 4, 6, 5, 10]$, the function should return 3, as fractions $1/2$, $2/4$, $3/6$ are all equal to each other.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array X is an integer within the range $[0..2,000,000,000]$;
- each element of array Y is an integer within the range $[1..2,000,000,000]$;
- arrays X and Y are of equal length.

RedSegment

There are N balls positioned in a row. Each of them is either red or white. In one move we can swap two adjacent balls. We want to arrange all the red balls into a consistent segment. What is the minimum number of swaps needed?

Write a function:

```
def solution(S)
```

that, given string S of length N built from characters "R" and "W", representing red and white balls respectively, returns the minimum number of swaps needed to arrange all the red balls into a consistent segment. If the result exceeds 10^9 , return -1.

Examples:

1. Given $S = \text{"WRRWWR"}$, your function should return 2. We can move the last ball two positions to the left:

- "WRRWRW"
- "WRRRWW"

2. Given $S = \text{"WWRWWWRWR"}$, your function should return 4. We can move first and last red ball towards the middle one:

- "WWWRWWWR"
- "WWWWRWR"
- "WWWWRWR"
- "WWWWRRRW"

3. Given $S = \text{"WWW"}$, your function should return 0. There are no red balls to arrange into a segment.

4. Given S is "RW" repeated 100,000 times, your function should return -1. The minimum needed number of swaps is greater than 10^9 .

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..200,000];
- string S is made only of the characters 'R' and/or 'W'.

EraseAdjacentLetters

A string S consisting of the letters A, B, C and D is given. The string can be transformed either by removing a letter A together with an adjacent letter B, or by removing a letter C together with an adjacent letter D.

Write a function:

```
def solution(S)
```

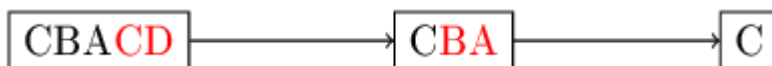
that, given a string S consisting of N characters, returns any string that:

- can be obtained from S by repeatedly applying the described transformation, and
- cannot be further transformed.

If at some point there is more than one possible way to transform the string, any of the valid transformations may be chosen.

Examples:

1. Given S = "CBACD", the function may return "C", because one of the possible sequences of operations is as follows:



2. Given S = "CABABD" the function may return an empty string, because one possible sequence of operations is:



3. Given string S = "ACBDACBD" the function should return "ACBDACBD", because no operation can be applied to string S.

Write an efficient algorithm for the following assumptions:

- the length of string S is within the range [0..250,000];
- string S is made only of the following characters: 'A', 'B', 'C' and/or 'D'.

FourDigitsPath

You are given a board of N rows and M columns. Each field of the board contains a single digit (0-9).

You want to find a path consisting of four neighboring fields. Two fields are neighboring if they share a common side. Also, the fields in your path should be distinct (you can't visit the same field twice).

The four digits of your path, in the order in which you visit them, create an integer. What is the biggest integer that you can achieve in this way?

Write a function

```
def solution(Board)
```

that, given the board represented as a matrix of integers consisting of N rows and M columns, returns the biggest integer that you can achieve when concatenating the values in a path of length four.

Examples:

1. Given the following board ($N=3$, $M=5$):

9	1	1	0	7
1	0	2	1	0
1	9	1	1	0

the function should return 9121. You can choose either of the following paths (the first field is denoted by red):

9	1	1	0	7
1	0	2	1	0
1	9	1	1	0

or

9	1	1	0	7
1	0	2	1	0
1	9	1	1	0

2. Given the following board ($N=3$, $M=3$):

1	1	1
1	3	4
1	4	3

the function should return 4343. The best path is:

1	1	1
1	3	4
1	4	3

3. Given the following board (N=1, M=5):

0	1	5	0	0
---	---	---	---	---

the function should return 1500. The best path is:

0	1	5	0	0
---	---	---	---	---

Write an efficient algorithm for the following assumptions:

- N and M are integers within the range [1..100];
- each element of matrix Board is an integer within the range [0..9];
- there exist a path of length 4 which doesn't start with 0.

CandyShops

There are two candy shops, both of which have N candies for sale. The first shop sells each candy for one golden coin; the second shop sells each candy for one silver coin. You have N/2 golden coins and N/2 silver ones (N is even).

Each candy is of some particular type (not necessarily unique), represented by an integer. What is the maximum number of different types of candy that you can buy?

Write a function:

```
def solution(A, B)
```

that, given two arrays A and B of N integers, representing types of candies in the first and second shop respectively, returns the maximum possible number of different types of candy that you can buy.

For example, given:

```
A = [1, 2, 3, 4]
```

B = [3, 3, 3, 7]

the function should return 4. You can, for example, buy candies of types 1, 2 in the first shop and then candies of types 3, 7 in the second shop.

Given:

A = [2, 2, 2, 2, 2, 2]

B = [7, 4, 2, 5, 1, 2]

the function should also return 4. In the first shop you can only buy candies of type 2, but then you can buy three candies of other types (for example, 7, 4 and 5) in the second shop.

Write an efficient algorithm for the following assumptions:

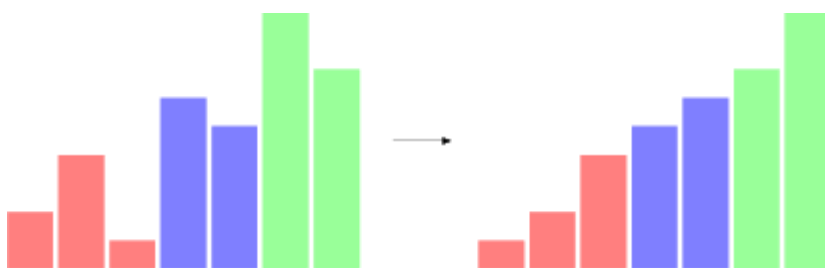
- N is an integer within the range [2..100,000];
- N is even;
- each element of arrays A and B is an integer within the range [1..1,000,000,000].

ArraySlicing

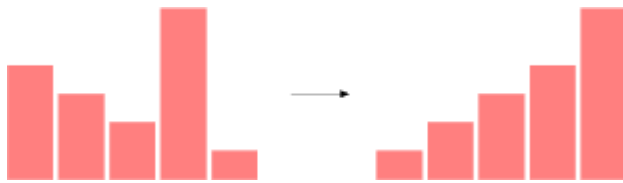
We are given an array A consisting of N distinct integers. We would like to sort array A into ascending order using a simple algorithm. First, we divide it into one or more slices (a slice is a contiguous subarray). Then we sort each slice. After that, we join the sorted slices in the same order. Write a function solution that returns the maximum number of slices for which the algorithm will return a correctly sorted array.

Examples:

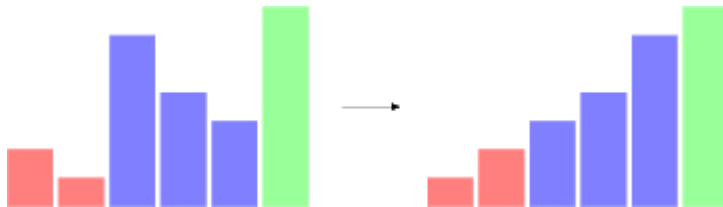
1. Given A = [2, 4, 1, 6, 5, 9, 7], the function should return 3. The array can be split into three slices: [2, 4, 1], [6, 5] and [9, 7]. Then, after sorting each slice and joining them together, the whole array will be sorted into ascending order.



2. Given A = [4, 3, 2, 6, 1], the function should return 1. The array cannot be split into smaller slices; it has to be sorted all at once.



3. Given $A = [2, 1, 6, 4, 3, 7]$, the function should return 3.



Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of array A is an integer within the range $[1..1,000,000,000]$;
- the elements of A are all distinct.

Basketballs

Tom works in a warehouse. A billion (1,000,000,000) boxes are arranged in a row. They are numbered from one to one billion (from left to right). Some boxes contain a basketball (at most one basketball in each box). In total, there are N basketballs.

Tom wants to organize the warehouse. He would like to see all the basketballs arranged next to each other (they should occupy a consistent interval of boxes). In one move, Tom can take one basketball and move it into an empty box. What is the minimum number of moves needed to organize the basketballs in the warehouse?

Write a function:

```
def solution(A)
```

that, given an array A containing N integers, denotes the positions of the basketballs (the numbers of the boxes in which they are placed) and returns the minimum number of moves needed to organize the basketballs in the warehouse.

For example, given: $A = [6, 4, 1, 7, 10]$, your function should return 2 because the minimum number of moves needed to arrange all basketballs next to each other is 2. There are several ways to do it. For example, you could move the ball from the first box to the fifth, and the ball from the tenth box to the eighth. You could also move the ball from the first box to the fifth, and the ball from the tenth box to the third instead. In any case, you need at least two moves.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..50,000];
- each element of array A is an integer within the range [1..1,000,000,000];
- numbers in array A are pairwise distinct.

OptimizingFarthestDifferent

You are given an implementation of a function:

```
def solution(A)
```

which accepts as input a non-empty array A consisting of N integers.

The function works slowly on large input data and the goal is to optimize it so as to achieve better time and/or space complexity. The optimized function should return the same result as the given implementation for every input that satisfies the assumptions.

For example, given array A such that:

```
A[0] = 4
A[1] = 6
A[2] = 2
A[3] = 2
A[4] = 6
A[5] = 6
A[6] = 4
```

the function returns 5.

Also, for given array A such that:

```
A[0] = 0
A[1] = 1
...
A[74999] = 74999
A[75000] = 75000
```

in other words, $A[K] = K$ for each K ($0 \leq K \leq 75,000$), the given implementation works too slow, but the function would return 75,000.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each element of array A is an integer within the range $[-1,000,000,000..1,000,000,000]$.

The original code is:

```
def solution(A):
    N = len(A)
```



```

result = 0
for i in range(N):
    for j in range(i, N):
        if A[i] != A[j]:
            result = max(result, j - i)
    return result

```

MaxBiValuedSlice

We call an array *bi-valued* if it contains at most two different numbers.

For example: [4, 5, 5, 4, 5] is bi-valued (it contains two different numbers: 4 and 5), but [1, 5, 4, 5] is not bi-valued (it contains three different numbers: 1, 4 and 5).

What is the length of the longest bi-valued slice (continuous fragment) in a given array A?

Write a function:

```
def solution(A)
```

that, given an array A consisting of N integers, returns the length of the longest bi-valued slice in A.

Examples:

1. Given A = [4, 2, 2, 4, 2], the function should return 5, because the whole array is bi-valued.
2. Given A = [1, 2, 3, 2], the function should return 3. The longest bi-valued slice is [2, 3, 2].
3. Given A = [0, 5, 4, 4, 5, 12], the function should return 4. The longest bi-valued slice is [5, 4, 4, 5].
4. Given A = [4, 4], the function should return 2.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each element of array A is an integer within the range [-1,000,000,000..1,000,000,000].

Times17DecDigitsSum

A non-empty array A consisting of N integers is given. This array contains a decimal representation of a number V, i.e. element A[K] contains the K-th least significant digit of the decimal representation of V.

For example, array A such that:

A[0] = 3
A[1] = 5
A[2] = 1

represents the number $V = 153$.

Write a function:

```
def solution(A)
```

that, given an array A consisting of N integers specifying a decimal representation of a number V, returns the sum of the digits in the decimal representation of the number $17 \cdot V$.

For example, given array A such that:

A[0] = 3
A[1] = 5
A[2] = 1

the function should return 9, because:

- array A represents the number 153,
- $17 \cdot 153 = 2601$,
- the sum of the digits in the decimal representation of 2601 is $2 + 6 + 0 + 1 = 9$.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range $[1..1,000,000]$;
- each element of array A is an integer within the range $[0..9]$.

IntegerReductionCount

You are given a string S of length N which encodes a non-negative number V in a binary form.

Two types of operations may be performed on it to modify its value:

- if V is odd, subtract 1 from it;
- if V is even, divide it by 2.

These operations are performed until the value of V becomes 0.

For example, if string S = "011100", its value V initially is 28. The value of V would change as follows:

- V = 28, which is even: divide by 2 to obtain 14;
- V = 14, which is even: divide by 2 to obtain 7;
- V = 7, which is odd: subtract 1 to obtain 6;
- V = 6, which is even: divide by 2 to obtain 3;
- V = 3, which is odd: subtract 1 to obtain 2;

- $V = 2$, which is even: divide by 2 to obtain 1;
- $V = 1$, which is odd: subtract 1 to obtain 0.

Seven operations were required to reduce the value of V to 0.

Write a function:

```
def solution(S)
```

that, given a string S consisting of N characters containing a binary representation of the initial value V , returns the number of operations after which its value will become 0.

Examples:

1. Given $S = "011100"$, the function should return 7. String S represents the number 28, which becomes 0 after seven operations, as explained above.

2. Given $S = "111"$, the function should return 5. String S encodes the number $V = 7$. Its value will change over the following five operations:

- $V = 7$, which is odd: subtract 1 to obtain 6;
- $V = 6$, which is even: divide by 2 to obtain 3;
- $V = 3$, which is odd: subtract 1 to obtain 2;
- $V = 2$, which is even: divide by 2 to obtain 1;
- $V = 1$, which is odd: subtract 1 to obtain 0.

3. Given $S = "1111010101111"$, the function should return 22.

4. Given string S consisting of "1" repeated 400,000 times, the function should return 799,999.

Write an efficient algorithm for the following assumptions:

- string S is made only of the characters '0' and/or '1';
- N , which is the length of string S , is an integer within the range $[1..1,000,000]$;
- the binary representation is big-endian, i.e. the first character of string S corresponds to the most significant bit;
- the binary representation may contain leading zeros.

OnesInArithSeq

Write a function:

```
def solution(N)
```

that, given an integer N , returns the number of times the digit 1 occurs in decimal representations of all positive integers not exceeding N .

For example, given $N = 13$ the function should return 6, because:

- all the positive integers that do not exceed 13 are 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 and 13;
- digit 1 occurs six times altogether: once in number 1, once in number 10, twice in number 11, once in number 12 and once in number 13.

Write an efficient algorithm for the following assumptions:

- N is an integer within the range [0..100,000,000].